Dipartimento di Ingegneria Informatica, Modellistica, Elettronica e Sistemistica

---

Master Degree in "Robotics and Automation Engineering"

Course: Programmazione dei sistemi Tempo-Reale e Distribuiti

MODELLAZIONE DI UN CENTRO DI CHIRURGIA E RIABILITAZIONE ROBOTICA

Student

Ludovica Perroni
269200

Professor

Francesco Pupo

Academic Year 2025-2026

Indice

1 Introduzione **3**

2 Scenario e requisiti di sistema **5**

3 Modellazione mediante Reti di Petri **8**
3.1 Modello base 8
3.2 Rappresentazione matriciale del modello base 10
3.3 Introduzione delle classi di pazienti 12
3.4 Rappresentazione matriciale del modello con un paziente PL e un paziente EM 14
3.5 Estensione a più pazienti 16
3.6 Nota sulla rete multi-paziente e sulle matrici 17
3.7 Proprietà del modello 21

4 Modellazione mediante Automi Temporizzati **22**
4.1 Richiami teorici 22
4.2 Struttura del modello 23
4.3 Flusso del paziente 23
4.3.1 Comportamento paziente programmato (PL) 24
4.3.2 Comportamento paziente in emergenza (EM) 25
4.4 Clock, guardie e invarianti nel modello 25
4.4.1 Clock 25
4.4.2 Guardie 26
4.4.3 Invarianti 26
4.5 Gestione delle risorse 26
4.6 Sincronizzazione e cicli di attesa 26
4.6.1 Canali utilizzati 26
4.6.2 Ciclo tra MultiChoices e WaitingSurgery 27
4.6.3 Ciclo tra MultiChoicesRehab e WaitingRehab 27
4.6.4 Interpretazione del modello 28
4.7 Verifica tramite query 28
4.7.1 Mutua esclusione 28
4.7.2 Vincoli temporali 29
4.7.3 Correttezza globale 31
4.8 Analisi dettagliata della simulazione 32
4.8.1 Configurazione iniziale 33
4.8.2 Evoluzione del sistema e competizione per le risorse 35
4.8.3 Fase di riabilitazione 36
4.8.4 Completamento del sistema 37

4.8.5 Osservazioni principali . . . 38
4.9 Conclusione . . . 38

5 Scheduling real-time del sistema **39**
5.1 Modellazione del sistema come problema real-time . . . 39
5.2 Politiche di scheduling . . . 40
5.2.1 Fixed Priority Scheduling (FPS) . . . 40
5.2.2 Earliest Deadline First (EDF) . . . 40
5.2.3 Least Slack Time (LST) . . . 40
5.2.4 Non-Preemptive Scheduling . . . 40
5.3 Confronto tra le politiche . . . 41
5.4 Scelta della politica di scheduling . . . 41
5.5 Analisi della schedulabilità . . . 41
5.5.1 Condizione per EDF . . . 41
5.5.2 Confronto con il bound di Liu & Layland . . . 42
5.5.3 Interpretazione nel contesto del sistema . . . 42
5.6 Esempio numerico di scheduling . . . 42
5.6.1 Caso di overload . . . 43
5.7 Esempio comparativo su tempi di attesa . . . 46
5.7.1 Boxplot dei tempi di attesa . . . 47
5.7.2 CDF dei tempi di attesa . . . 48
5.8 Accesso alle risorse condivise . . . 49
5.9 Priority Inversion Problem . . . 50
5.9.1 Esempio con tre task . . . 50
5.10 Priority Inheritance Protocol . . . 50
5.10.1 Effetto sull'esempio . . . 51
5.11 Priority Ceiling Protocol . . . 51
5.11.1 Effetto sull'esempio . . . 52
5.12 Applicazione al sistema ospedaliero . . . 52
5.13 Scelta del protocollo . . . 52

6 Conclusione **54**

Capitolo 1

Introduzione

I sistemi robotici moderni, in particolare in ambito **medico-ospedaliero**, richiedono:

- elevata **affidabilità**,
- corretta gestione delle **risorse condivise**,
- rispetto di **vincoli temporali stringenti**.

In tali contesti, più attività vengono eseguite in **concorrenza** e competono per un numero limitato di risorse, rendendo la progettazione del sistema complessa dal punto di vista della **sicurezza** e della **correttezza operativa**.

I sistemi concorrenti sono caratterizzati dalla presenza di più entità che evolvono simultaneamente e interagiscono tra loro, condividendo risorse e richiedendo meccanismi di sincronizzazione. In tali scenari, modelli sequenziali tradizionali risultano spesso inadeguati, rendendo necessario l'utilizzo di formalismi più espressivi per descrivere il comportamento globale del sistema.

Negli ultimi anni, la **robotica in ambito sanitario** ha migliorato:

- la **precisione** degli interventi chirurgici,
- l'**efficacia** dei percorsi di riabilitazione.

Tuttavia, essa introduce nuove sfide legate alla **coordinazione** tra risorse tecnologiche e umane.

Obiettivo del progetto
Il presente lavoro ha come obiettivo la **modellazione e analisi** di un centro ospedaliero dotato di risorse condivise, quali:

- robot chirurgico,
- robot di riabilitazione,
- personale sanitario,
- sala operatoria.

Tali risorse devono essere **allocate correttamente** per evitare comportamenti:

- non sicuri,
- inefficienti.

Caratteristiche del sistema
Il problema presenta caratteristiche tipiche dei sistemi:

- **concorrenti** → gestione della mutua esclusione,
- **real-time** → rispetto di **deadline** e tempi di attesa.

In particolare, nei sistemi real-time la correttezza non dipende unicamente dal risultato delle operazioni, ma anche dal tempo in cui tali operazioni vengono eseguite. Il mancato rispetto di una deadline può infatti compromettere il corretto funzionamento del sistema.

Metodologia adottata
Per affrontare il problema vengono utilizzati diversi formalismi:

- **Reti di Petri (PIPE)** → modellazione della concorrenza e delle risorse,
- **Automi temporizzati (UPPAAL)** → gestione e verifica dei vincoli temporali,
- **Scheduling real-time** → analisi dell'allocazione delle risorse e rispetto delle scadenze.

Ciascun formalismo consente di analizzare il sistema da un punto di vista differente: strutturale, temporale e decisionale, fornendo una visione completa del comportamento complessivo.

Obiettivo finale
Dimostrare come l'integrazione di **metodi formali** e tecniche di **analisi real-time** permetta di:

- modellare sistemi complessi,
- analizzarne il comportamento,
- verificarne la correttezza,

in contesti **safety-critical**.

Capitolo 2

Scenario e requisiti di sistema

Il sistema considerato è un centro ospedaliero dotato di risorse limitate e condivise per la chirurgia robotica e la riabilitazione post-operatoria.

Risorse del sistema
Le principali risorse disponibili sono:

- sala operatoria,
- robot chirurgico,
- chirurgo,
- robot di riabilitazione.

Tali risorse sono **condivise** e soggette a vincoli di **mutua esclusione**, in quanto non possono essere assegnate simultaneamente a più pazienti.

Flusso operativo
Il percorso di ciascun paziente è modellato come una sequenza di fasi:

1. arrivo e triage,
2. accesso alla fase chirurgica,
3. riabilitazione post-operatoria,
4. uscita dal sistema.

Questo flusso descrive l'evoluzione del paziente all'interno del sistema e mette in evidenza la dipendenza dalle risorse disponibili.

Schema del flusso del paziente
La sequenza operativa descritta è rappresentata nello schema concettuale in Figura 2.1, che evidenzia le principali fasi del percorso del paziente all'interno del sistema.

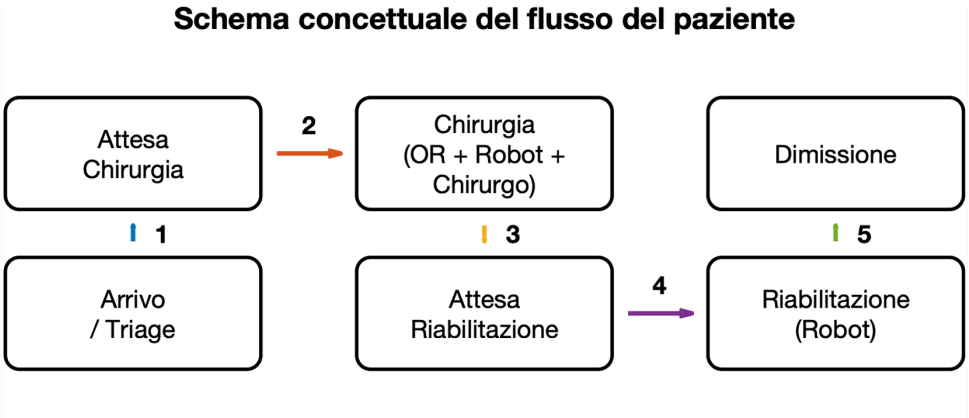


Figura 2.1: Schema concettuale del flusso del paziente nel sistema ospedaliero.

Classi di pazienti
Si distinguono due tipologie:

- pazienti programmati (PL),
- pazienti in emergenza (EM).

I pazienti EM hanno **priorità** nell'accesso alle risorse, introducendo un meccanismo di gestione differenziata delle richieste.

Vincoli temporali
Il sistema è soggetto ai seguenti vincoli:

- ogni paziente EM deve iniziare l'intervento entro T_{surg_max};
- la riabilitazione ha durata prefissata T_{rehab}.

Il vincolo su T_{surg_max} rappresenta una **deadline real-time**, che impone un limite massimo sul tempo di attesa prima dell'accesso alla risorsa critica.

Modello astratto
Il sistema può essere interpretato come:

- un insieme di **task concorrenti** (pazienti),
- che competono per **risorse limitate**,
- soggetti a **vincoli temporali**.

In tale contesto, il comportamento del sistema emerge dall'interazione tra le diverse entità e dalla disponibilità delle risorse, rendendo necessaria una modellazione formale per analizzarne correttamente le dinamiche.

Requisiti
Il sistema deve garantire:

- mutua esclusione nell'accesso alle risorse,
- rispetto delle priorità EM/PL,

- rispetto delle deadline,
- assenza di deadlock e conflitti.

Tali requisiti costituiscono i criteri fondamentali per la successiva modellazione e verifica del sistema, e guideranno l'analisi nei capitoli successivi.

Capitolo 3

Modellazione mediante Reti di Petri

Il sistema descritto nel Capitolo 2 viene modellato mediante reti di Petri, al fine di rappresentare in modo formale la concorrenza tra i pazienti e la gestione delle risorse condivise.

Una rete di Petri è formalmente definita come una tupla (P, T, F, M_0), dove:

- P è l'insieme dei posti,
- T è l'insieme delle transizioni,
- F è l'insieme degli archi,
- M_0 è la marcatura iniziale.

La dinamica del sistema è descritta dall'evoluzione della marcatura, ovvero dal numero di token presenti nei posti.

In questo contesto:

- i **posti** rappresentano stati del sistema o risorse,
- le **transizioni** rappresentano eventi,
- i **token** rappresentano i pazienti o la disponibilità delle risorse.

Una transizione è abilitata se tutti i posti di ingresso contengono un numero sufficiente di token. Lo scatto della transizione modifica la marcatura della rete, rappresentando l'evoluzione del sistema.

Il modello è stato sviluppato secondo un approccio incrementale, partendo da una configurazione base e introducendo progressivamente maggiore complessità.

3.1 Modello base

La Figura 3.1 rappresenta il modello base della rete di Petri, che descrive il comportamento di un singolo paziente.

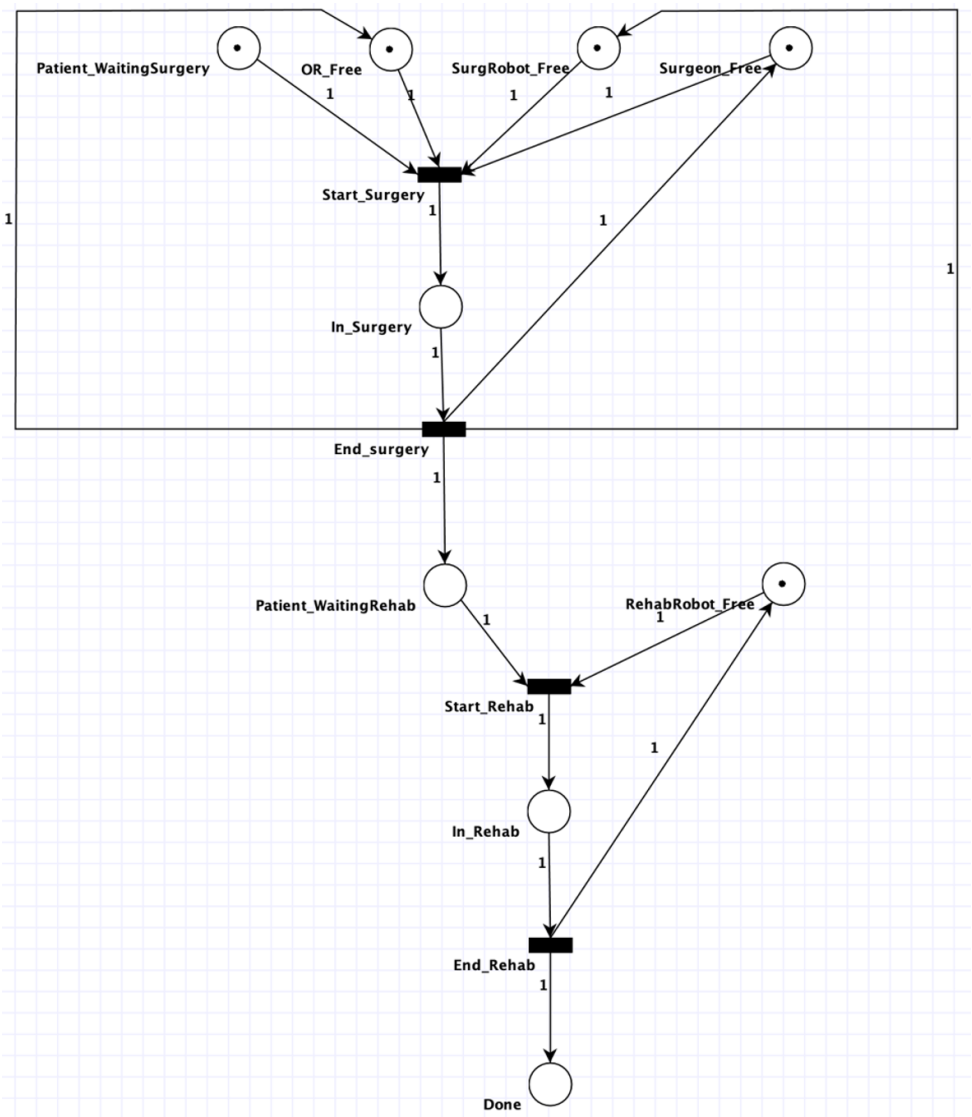


Figura 3.1: Modello base della rete di Petri.

Il paziente è rappresentato da un token che attraversa i posti:

- `Patient_WaitingSurgery`,
- `In_Surgery`,
- `Patient_WaitingRehab`,
- `In_Rehab`.

Le risorse sono modellate tramite i posti:

- `OR_Free`,
- `SurgRobot_Free`,
- `Surgeon_Free`,
- `RehabRobot_Free`.

I posti rappresentano il tipo di risorsa, mentre il numero di token in tali posti rappresenta la disponibilità.

Le transizioni `Start_Surgery` e `Start_Rehab` sono abilitate solo se:

- è presente un paziente,
- tutte le risorse richieste sono disponibili.

Durante l'esecuzione, i token delle risorse vengono consumati e successivamente rilasciati al termine delle operazioni, garantendo la mutua esclusione.

Descrizione del flusso

Il funzionamento della rete può essere descritto come segue:

1. il paziente entra nel sistema e si trova nello stato di attesa per la chirurgia;
2. quando tutte le risorse sono disponibili, la transizione di inizio intervento si abilita;
3. il paziente entra nello stato di intervento chirurgico e le risorse vengono occupate;
4. al termine dell'intervento, le risorse vengono rilasciate e il paziente passa alla fase di riabilitazione;
5. il paziente accede alla riabilitazione se la risorsa è disponibile;
6. al termine della riabilitazione, il paziente esce dal sistema.

3.2 Rappresentazione matriciale del modello base

Per la rete di Petri di Figura 3.1, si considera il seguente ordinamento dei posti:

$$
\begin{aligned}
p_1 &= \texttt{Patient_WaitingSurgery},\\
p_2 &= \texttt{OR_Free},\\
p_3 &= \texttt{SurgRobot_Free},\\
p_4 &= \texttt{Surgeon_Free},\\
p_5 &= \texttt{In_Surgery},\\
p_6 &= \texttt{Patient_WaitingRehab},\\
p_7 &= \texttt{RehabRobot_Free},\\
p_8 &= \texttt{In_Rehab},\\
p_9 &= \texttt{Done}.
\end{aligned}
$$

Le transizioni sono:

$$
\begin{aligned}
t_1 &= \texttt{Start_Surgery},\\
t_2 &= \texttt{End_Surgery},\\
t_3 &= \texttt{Start_Rehab},\\
t_4 &= \texttt{End_Rehab}.
\end{aligned}
$$

La marcatura iniziale è:

$$M_0^{(\text{base})} = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$$

La marcatura iniziale rappresenta la configurazione di partenza del sistema, cioè come sono distribuiti i token nei posti all'istante iniziale.

Nel caso in esame, la marcatura iniziale indica:

- la presenza di un paziente in attesa di intervento chirurgico;
- la disponibilità di tutte le risorse chirurgiche (sala operatoria, robot chirurgico e chirurgo);
- la disponibilità del robot riabilitativo;
- l'assenza di pazienti nelle fasi di chirurgia, riabilitazione e completamento.

Le matrici di incidenza consentono di descrivere in modo formale la dinamica della rete. In particolare, esse modellano il flusso dei token tra i posti al verificarsi delle transizioni.

La matrice di incidenza in ingresso è:

$$I^-_{\text{base}} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Essa indica, per ciascuna transizione, quali posti forniscono i token necessari allo scatto: ogni elemento rappresenta quindi un **consumo di risorse o pazienti**.

La matrice di incidenza in uscita è:

$$I^+_{\text{base}} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Essa descrive invece la produzione di token a seguito dello scatto delle transizioni, cioè la **liberazione delle risorse e l'avanzamento del paziente nel percorso clinico**.

La matrice di incidenza complessiva è quindi:

$$I_{\text{base}} = I^{+}_{\text{base}} - I^{-}_{\text{base}} = \begin{bmatrix} -1 & 0 & 0 & 0 \\ -1 & 1 & 0 & 0 \\ -1 & 1 & 0 & 0 \\ -1 & 1 & 0 & 0 \\ 1 & -1 & 0 & 0 \\ 0 & 1 & -1 & 0 \\ 0 & 0 & -1 & 1 \\ 0 & 0 & 1 & -1 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Questa matrice sintetizza l'effetto netto delle transizioni sulla marcatura del sistema, evidenziando contemporaneamente consumi e produzioni di token.

L'evoluzione della rete è descritta dalla relazione:

$$M' = M + I_{\text{base}}\,\sigma$$

dove M è la marcatura corrente, M' è la marcatura successiva e σ è il vettore di scatto delle transizioni.

Dal punto di vista interpretativo:

- la prima colonna descrive l'avvio della chirurgia: il paziente lascia la coda di attesa, le risorse chirurgiche vengono occupate e il token entra in `In_Surgery`;
- la seconda colonna rappresenta il termine della chirurgia, con il rilascio delle risorse e il trasferimento del paziente verso la fase di riabilitazione;
- la terza colonna modella l'avvio della riabilitazione, con l'occupazione del robot riabilitativo;
- la quarta colonna descrive il completamento del percorso clinico, con il rilascio delle risorse e il passaggio del paziente nello stato `Done`.

In questo modo, la rappresentazione matriciale consente di passare da una descrizione grafica ad un modello matematico rigoroso, utile per l'analisi formale del sistema e per la verifica delle sue proprietà dinamiche.

3.3 Introduzione delle classi di pazienti

La Figura 3.2 estende il modello base introducendo due classi di pazienti:

- pazienti programmati (PL),
- pazienti in emergenza (EM).

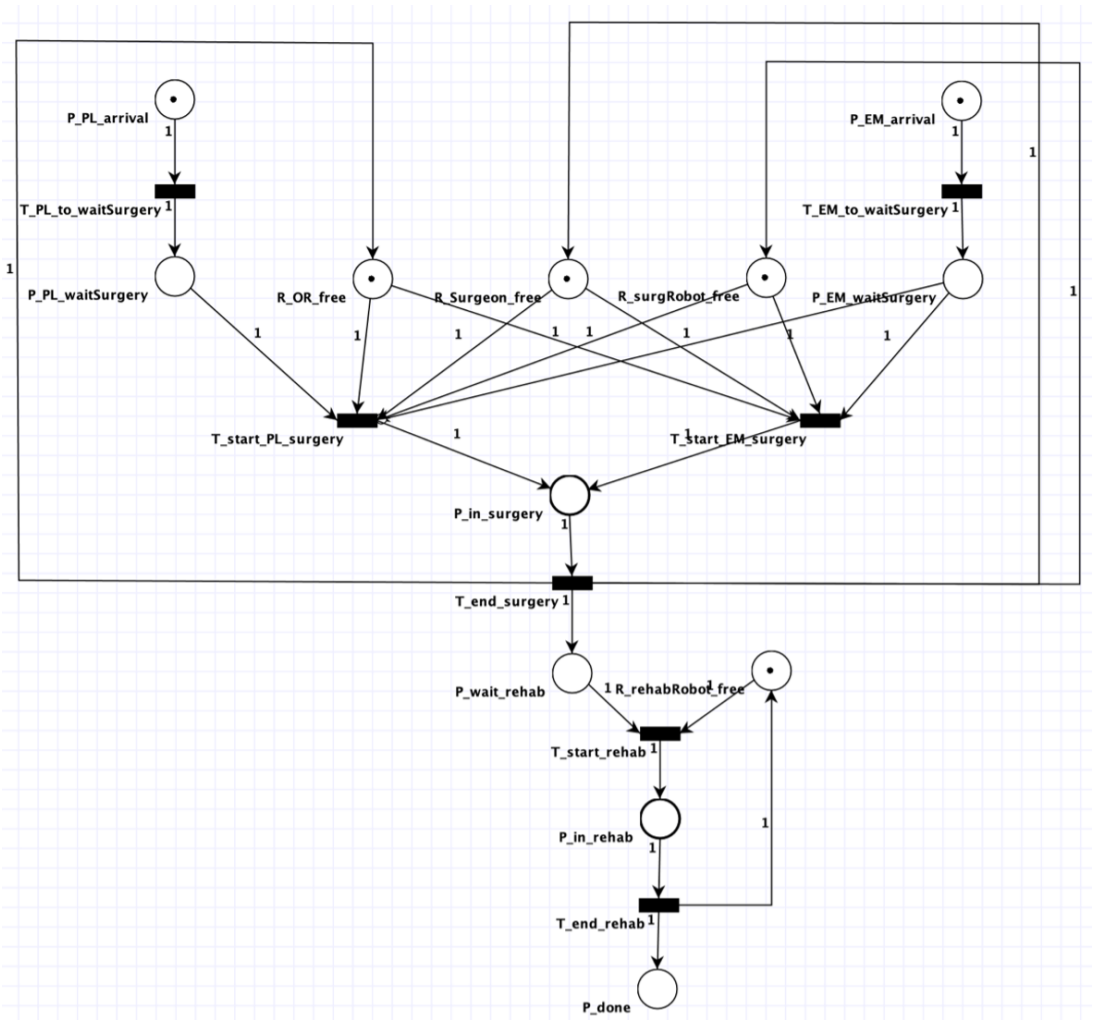


Figura 3.2: Modello con un paziente PL e un paziente EM.

I due flussi sono modellati separatamente attraverso i posti:

- `P_PL_waitSurgery`,
- `P_EM_waitSurgery`.

Entrambi i pazienti competono per le stesse risorse condivise:

- `R_OR_free`,
- `R_Surgeon_free`,
- `R_surgRobot_free`.

Le transizioni:

- `T_start_PL_surgery`,
- `T_start_EM_surgery`

modellano l'inizio dell'intervento per le due categorie.

La separazione dei flussi consente di rappresentare la competizione per le risorse e l'introduzione della priorità dei pazienti EM.

3.4 Rappresentazione matriciale del modello con un paziente PL e un paziente EM

Per la rete di Figura 3.2, si considera il seguente ordinamento dei posti:

$$
\begin{aligned}
p_1 &= \texttt{P_PL_arrival}, \\
p_2 &= \texttt{P_PL_waitSurgery}, \\
p_3 &= \texttt{P_EM_arrival}, \\
p_4 &= \texttt{P_EM_waitSurgery}, \\
p_5 &= \texttt{R_OR_free}, \\
p_6 &= \texttt{R_Surgeon_free}, \\
p_7 &= \texttt{R_surgRobot_free}, \\
p_8 &= \texttt{P_in_surgery}, \\
p_9 &= \texttt{P_wait_rehab}, \\
p_{10} &= \texttt{R_rehabRobot_free}, \\
p_{11} &= \texttt{P_in_rehab}, \\
p_{12} &= \texttt{P_done}.
\end{aligned}
$$

Le transizioni sono:

$$
\begin{aligned}
t_1 &= \texttt{T_PL_to_waitSurgery}, \\
t_2 &= \texttt{T_EM_to_waitSurgery}, \\
t_3 &= \texttt{T_start_PL_surgery}, \\
t_4 &= \texttt{T_start_EM_surgery}, \\
t_5 &= \texttt{T_end_surgery}, \\
t_6 &= \texttt{T_start_rehab}, \\
t_7 &= \texttt{T_end_rehab}.
\end{aligned}
$$

Assumendo un paziente PL e un paziente EM inizialmente presenti, la marcatura iniziale è:

$$
M_0^{(2)} = \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \\ 1 \\ 1 \\ 1 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}
$$

La marcatura iniziale descrive una situazione in cui entrambi i pazienti (programmato e urgente) sono presenti nel sistema ma non ancora entrati nella coda chirurgica, mentre tutte le risorse risultano disponibili.

Le matrici di incidenza permettono di rappresentare formalmente il comportamento dinamico del sistema, evidenziando come i pazienti competono per le risorse condivise.

La matrice di incidenza in ingresso è:

$$I^{-}_{(2)} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Essa rappresenta il **consumo di token**, ovvero:

- l'ingresso dei pazienti nelle rispettive code chirurgiche;
- l'occupazione simultanea delle risorse (sala operatoria, chirurgo e robot chirurgico) durante l'avvio della chirurgia;
- l'utilizzo del robot riabilitativo nella fase di riabilitazione.

La matrice di incidenza in uscita è:

$$I^{+}_{(2)} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Essa descrive la **produzione di token**, cioè:

- il passaggio dei pazienti nelle fasi successive del percorso clinico;
- il rilascio delle risorse dopo l'intervento chirurgico;
- il completamento della riabilitazione.

La matrice di incidenza complessiva è quindi:

$$I_{(2)} = I^{+}_{(2)} - I^{-}_{(2)} = \begin{bmatrix} -1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & -1 & 0 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & -1 & 0 & 0 & 0 \\ 0 & 0 & -1 & -1 & 1 & 0 & 0 \\ 0 & 0 & -1 & -1 & 1 & 0 & 0 \\ 0 & 0 & -1 & -1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & -1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & -1 & 0 \\ 0 & 0 & 0 & 0 & 0 & -1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & -1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Anche in questo caso l'evoluzione del sistema è descritta dalla relazione:

$$M' = M + I_{(2)}\,\sigma$$

Dal punto di vista interpretativo:

- le prime due colonne descrivono l'ingresso dei pazienti PL ed EM nelle rispettive code chirurgiche;
- le colonne t_3 e t_4 modellano l'avvio della chirurgia per le due classi, evidenziando la competizione per le stesse risorse condivise;
- la colonna t_5 rappresenta il termine dell'intervento chirurgico, con rilascio delle risorse e passaggio alla fase riabilitativa;
- le ultime due colonne descrivono l'avvio e il completamento della riabilitazione.

Si osserva che la presenza di due classi di pazienti introduce un aspetto fondamentale del sistema: la **priorità clinica**. In particolare, il modello consente di rappresentare politiche di scheduling in cui i pazienti EM (emergenza) possono avere precedenza rispetto ai pazienti PL (programmati), pur condividendo le stesse risorse critiche.

La rappresentazione matriciale permette quindi non solo di descrivere l'evoluzione del sistema, ma anche di analizzare formalmente gli effetti delle politiche di priorità e della competizione per le risorse.

3.5 Estensione a più pazienti

La Figura 3.3 rappresenta l'estensione del modello a più pazienti.

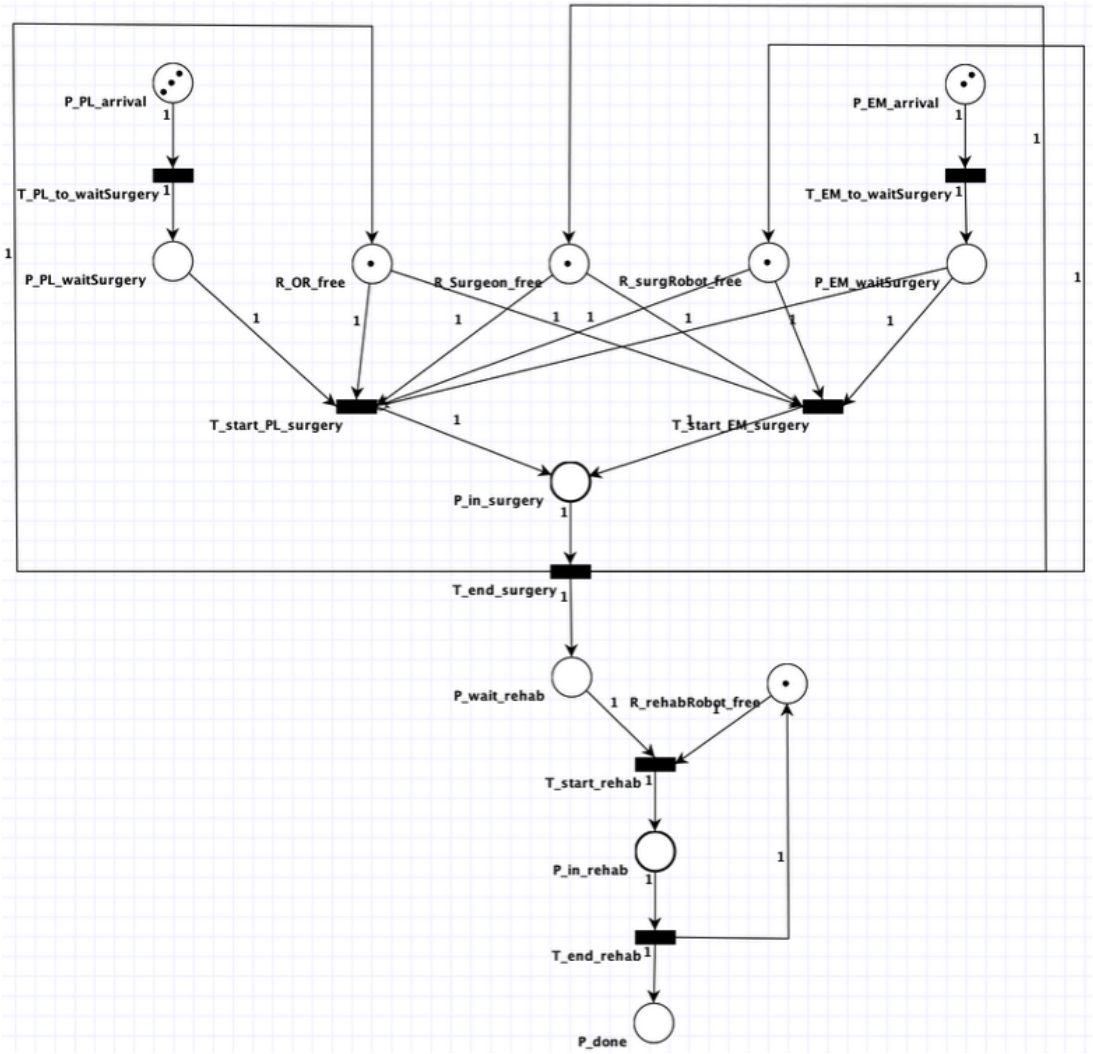


Figura 3.3: Modello con più pazienti PL ed EM.

La presenza di più token nei posti di arrivo rappresenta l'ingresso di più pazienti nel sistema.

I pazienti evolvono in modo concorrente e competono per l'accesso alle risorse condivise.

Questo modello permette di rappresentare condizioni realistiche e di analizzare:

- la gestione della concorrenza,
- l'utilizzo delle risorse,
- l'effetto delle priorità tra pazienti.

3.6 Nota sulla rete multi-paziente e sulle matrici

La rete mostrata in Figura 3.3 non introduce nuove connessioni strutturali rispetto al modello con due classi di pazienti, ma rappresenta una diversa configurazione iniziale caratterizzata da un numero maggiore di token.

In termini formali, la **struttura della rete** (posti, transizioni e archi) rimane invariata; di conseguenza, anche le matrici di incidenza $I^-_{(2)}$, $I^+_{(2)}$ e $I_{(2)}$ non subiscono modifiche. Questa proprietà è nota come **invarianza strutturale** della rete.

Riepilogando le matricic ottengo:

Forwards incidence matrix I^+

	T0	T1	T2	T3	T4	T5	T6
P0	0	0	0	0	0	0	0
P1	0	0	0	0	0	0	0
P10	0	0	0	0	0	1	0
P11	0	0	0	0	0	0	1
P2	1	0	0	0	0	0	0
P3	0	1	0	0	0	0	0
P4	0	0	0	0	1	0	0
P5	0	0	0	0	1	0	0
P6	0	0	0	0	1	0	0
P7	0	0	1	1	0	0	0
P8	0	0	0	0	1	0	0
P9	0	0	0	0	0	0	1

Figura 3.4: Matrice di ingresso.

Backwards incidence matrix I^-

	T0	T1	T2	T3	T4	T5	T6
P0	1	0	0	0	0	0	0
P1	0	1	0	0	0	0	0
P10	0	0	0	0	0	0	1
P11	0	0	0	0	0	0	0
P2	0	0	1	0	0	0	0
P3	0	0	0	1	0	0	0
P4	0	0	1	1	0	0	0
P5	0	0	1	1	0	0	0
P6	0	0	1	1	0	0	0
P7	0	0	0	0	1	0	0
P8	0	0	0	0	0	1	0
P9	0	0	0	0	0	1	0

Figura 3.5: Matrice di uscita.

Combined incidence matrix I

	T0	T1	T2	T3	T4	T5	T6
P0	-1	0	0	0	0	0	0
P1	0	-1	0	0	0	0	0
P10	0	0	0	0	0	1	-1
P11	0	0	0	0	0	0	1
P2	1	0	-1	0	0	0	0
P3	0	1	0	-1	0	0	0
P4	0	0	-1	-1	1	0	0
P5	0	0	-1	-1	1	0	0
P6	0	0	-1	-1	1	0	0
P7	0	0	1	1	-1	0	0
P8	0	0	0	0	1	-1	0
P9	0	0	0	0	0	-1	1

Figura 3.6: Matrice di incidenza.

Figura 3.7: Grafo di raggiungibilità.

Il grafo di raggiungibilità rappresenta tutte le possibili evoluzioni della rete di Petri a partire dalla marcatura iniziale.

Ogni nodo del grafo corrisponde a una marcatura raggiungibile, ovvero a una specifica configurazione del sistema in termini di distribuzione dei token nei posti, mentre gli archi rappresentano lo scatto delle transizioni che permettono il passaggio da uno stato all'altro.

L'elevato numero di nodi evidenzia la presenza di molteplici stati raggiungibili, dovuti alla concorrenza tra i pazienti e alla competizione per le risorse condivise.

Dall'analisi del grafo si osserva che:

- il sistema è **limitato (bounded)**, in quanto il numero di token nei posti non cresce indefinitamente;
- non si evidenziano stati di **deadlock**, poiché da ogni configurazione esiste almeno una transizione abilitata;
- esistono cammini che conducono a stati in cui i pazienti raggiungono il posto `Done`, garantendo la **raggiungibilità dello stato finale**.

Il grafo conferma quindi che il sistema è in grado di evolvere correttamente senza blocchi permanenti, gestendo in modo consistente la concorrenza e l'utilizzo delle risorse.

Ciò che cambia è esclusivamente la **marcatura iniziale**, che consente di modellare la presenza simultanea di più pazienti e quindi un maggiore livello di concorrenza nel sistema.

Indicando con n_{PL} il numero iniziale di pazienti programmati e con n_{EM} il numero iniziale di pazienti in emergenza, la marcatura iniziale può essere espressa come:

$$M_0^{(3)} = \begin{bmatrix} n_{PL} \\ n_{EM} \\ 0 \\ 0 \\ 1 \\ 1 \\ 1 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$$

Nel caso della rete mostrata in figura, sono presenti inizialmente 3 pazienti PL e 2 pazienti EM; pertanto, la marcatura iniziale diventa:

Marking

	P0	P1	P10	P11	P2	P3	P4	P5	P6	P7	P8	P9
Initial	3	3	0	0	0	0	1	1	1	0	0	1
Current	3	2	0	0	0	0	1	1	1	0	0	1

Enabled transitions

T0	T1	T2	T3	T4	T5	T6
yes	yes	no	no	no	no	no

Figura 3.8: Marcatura e transizioni abilitate iniziali.

Dal punto di vista interpretativo, l'aumento del numero di token comporta:

- una maggiore **concorrenza** tra pazienti per l'accesso alle risorse condivise;
- la possibile formazione di **code** nei posti di attesa;
- una maggiore rilevanza delle **politiche di priorità** tra pazienti EM e PL;
- un utilizzo più intensivo delle risorse critiche, come sala operatoria, chirurgo, robot chirurgico e robot riabilitativo.

Questa modellazione consente quindi di analizzare scenari più realistici senza modificare la struttura della rete, ma agendo esclusivamente sulla marcatura iniziale. Inoltre, evidenzia come la complessità del sistema emerga non dalla struttura della rete, bensì dal livello di carico e dalla competizione tra i pazienti per l'accesso alle risorse.

3.7 Proprietà del modello

Il modello è stato analizzato al fine di verificare alcune proprietà fondamentali delle reti di Petri, necessarie a garantire il corretto funzionamento del sistema e l'assenza di comportamenti indesiderati.

- **Boundedness**: il numero di token presenti in ciascun posto risulta limitato. In particolare, i posti che rappresentano le risorse sono **1-limitati**, in quanto modellano risorse con capacità unitaria (sala operatoria, chirurgo, robot). I posti associati ai pazienti possono invece contenere più token, ma sempre in modo finito, in funzione della marcatura iniziale.

- **Safeness**: i posti che rappresentano le risorse sono sicuri (safe), poiché non possono contenere più di un token. Ciò garantisce che una risorsa non venga mai assegnata contemporaneamente a più pazienti, evitando condizioni di utilizzo non realistico.

- **Liveness**: il sistema è privo di deadlock, in quanto esiste sempre almeno una sequenza di scatti che consente ai pazienti di proseguire lungo il percorso clinico. In particolare, il completamento della chirurgia e della riabilitazione permette il rilascio delle risorse, rendendo nuovamente abilitabili le transizioni successive.

- **Reachability**: a partire dalla marcatura iniziale, è possibile raggiungere stati in cui i pazienti completano il percorso terapeutico (posto `P_done`). Questo garantisce che il sistema sia in grado di evolvere fino a condizioni di completamento senza blocchi permanenti.

Nel complesso, queste proprietà assicurano che il modello sia coerente con il comportamento atteso del sistema ospedaliero: le risorse sono utilizzate correttamente, i pazienti non rimangono bloccati indefinitamente e il processo può evolvere fino al completamento per tutti i soggetti coinvolti.

Capitolo 4

Modellazione mediante Automi Temporizzati

Per analizzare i vincoli temporali del sistema ospedaliero, si utilizzano gli automi temporizzati, modellati tramite lo strumento UPPAAL.

Le reti di Petri permettono di descrivere la concorrenza e la gestione delle risorse, ma non includono esplicitamente il tempo. Gli automi temporizzati introducono invece clock, guardie e invarianti, consentendo di verificare proprietà temporali come il rispetto delle deadline e la durata delle attività.

Nel sistema considerato, questo è fondamentale per garantire che i pazienti inizino l'intervento chirurgico entro un tempo massimo e che l'intero processo si svolga correttamente.

4.1 Richiami teorici

Un automa temporizzato è definito come:

$$A = (L, l_0, C, E, I)$$

dove:

- L è l'insieme delle locazioni,
- l_0 è la locazione iniziale,
- C è l'insieme dei clock,
- E è l'insieme delle transizioni,
- I rappresenta gli invarianti.

Clock Sono variabili reali che aumentano nel tempo e misurano la durata delle attività o delle attese.

Guardie Sono condizioni sui clock che devono essere soddisfatte per attivare una transizione.

Invarianti Limitano il tempo massimo che il sistema può trascorrere in una locazione.

Evoluzione Il sistema evolve tramite:

- passaggio del tempo (delay),
- cambiamenti di stato (azioni).

4.2 Struttura del modello

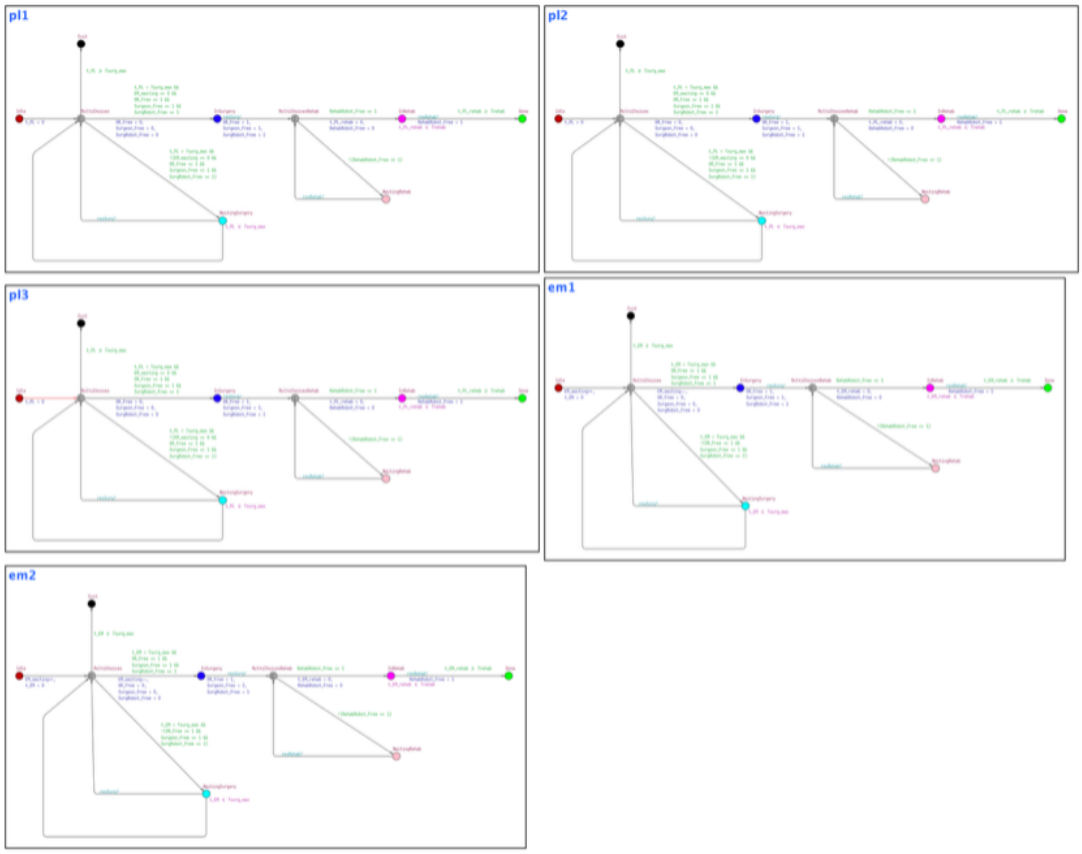


Figura 4.1: Sistema composto da più automi paziente.

Il sistema è composto da più automi, ciascuno rappresentante un paziente. Gli automi evolvono in parallelo e condividono le stesse risorse.

4.3 Flusso del paziente

Il comportamento di ciascun paziente segue un flusso ben definito:

1. **Idle** Il paziente entra nel sistema.
2. **MultiChoices (committed)** Il paziente deve immediatamente decidere cosa fare. Essendo una locazione committed:
   - il tempo non può avanzare,
   - la scelta è istantanea.

3. **WaitingSurgery** Il paziente attende che:
   - le risorse siano disponibili,
   - non venga superato il tempo massimo.
4. **InSurgery** Il paziente utilizza:
   - sala operatoria,
   - chirurgo,
   - robot chirurgico.
5. **WaitingRehab** Attesa per la riabilitazione.
6. **InRehab** Il paziente utilizza il robot di riabilitazione.
7. **Done** Il processo termina.

4.3.1 Comportamento paziente programmato (PL)

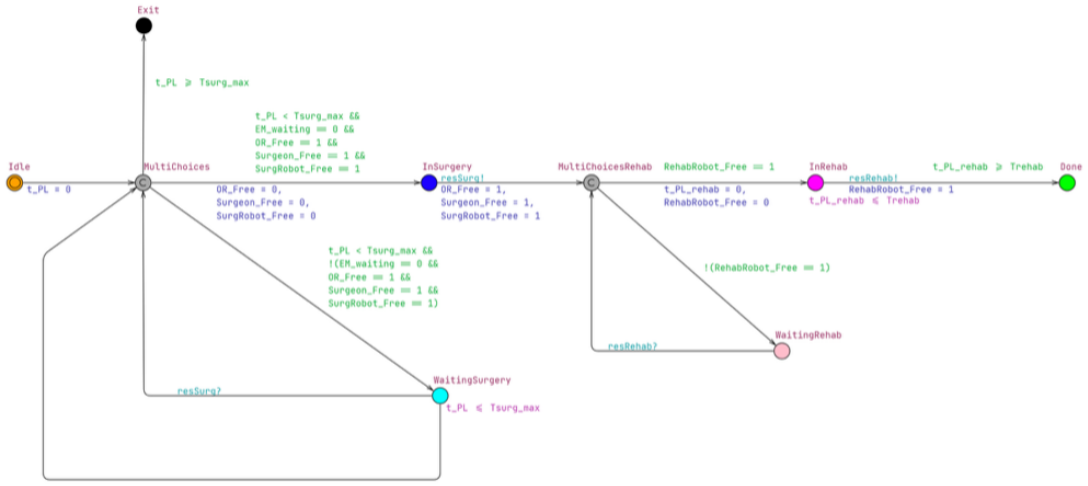


Figura 4.2: Automa temporizzato di un paziente programmato (PL).

Nel caso di pazienti programmati (PL), l'accesso alla chirurgia è subordinato a due condizioni:

- il tempo di attesa non deve superare la soglia T_{surg_max};
- non devono essere presenti pazienti in emergenza in attesa.

Infatti, come evidenziato nelle guardie:

- $t_{PL} < T_{surg_max}$ garantisce il rispetto della deadline;
- $EM_waiting = 0$ introduce una politica di priorità a favore dei pazienti in emergenza.

Questo implica che i pazienti PL possono essere ritardati in presenza di emergenze, evidenziando un comportamento non prioritario.

4.3.2 Comportamento paziente in emergenza (EM)

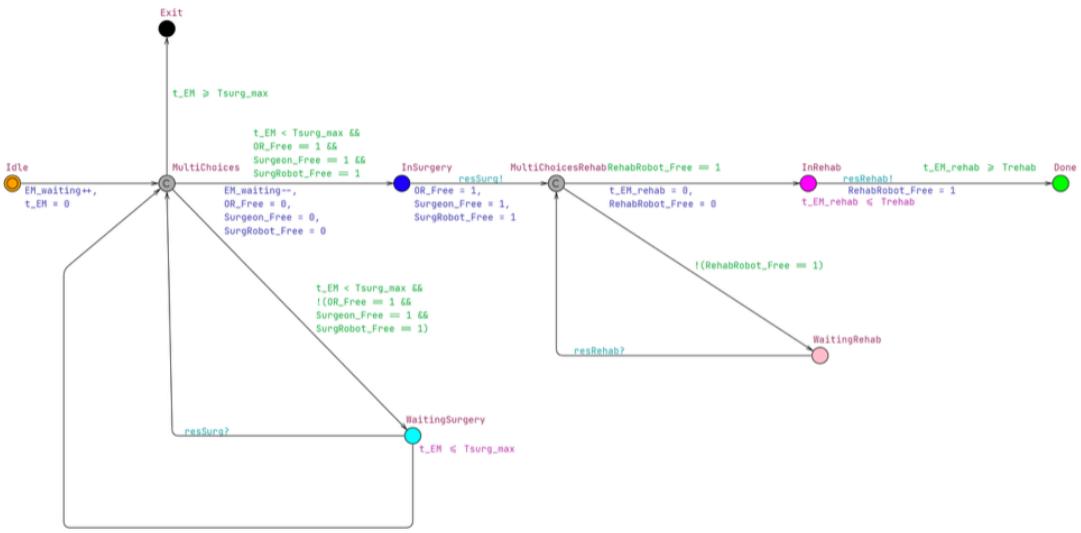


Figura 4.3: Automa temporizzato di un paziente in emergenza (EM).

Nel caso di pazienti in emergenza (EM), il comportamento è differente e prioritario.

Le guardie mostrano che:

- il paziente EM può accedere alla chirurgia appena le risorse sono disponibili;
- non è richiesto il controllo sulla presenza di altri pazienti in attesa.

In particolare, la condizione:

$$t_{EM} < T_{surg_max}$$

garantisce il rispetto del vincolo temporale, mentre l'assenza del vincolo su $EM_waiting$ evidenzia la priorità assoluta.

Questo modello implementa quindi una politica:

$$\mathbf{EM} \gg \mathbf{PL}$$

ovvero i pazienti in emergenza hanno precedenza rispetto a quelli programmati.

4.4 Clock, guardie e invarianti nel modello

Nel modello, i clock hanno un significato fisico preciso.

4.4.1 Clock

Ogni paziente ha clock che:

- misurano il tempo di attesa per la chirurgia,
- misurano la durata della riabilitazione.

4.4.2 Guardie

Esempio fondamentale:

$$t \leq T_{surg_max}$$

Significa che:

- il paziente può accedere alla chirurgia solo entro il tempo massimo.

4.4.3 Invarianti

Gli invarianti limitano il tempo trascorso in uno stato, evitando attese infinite.

4.5 Gestione delle risorse

Le risorse sono modellate tramite variabili globali:

- valore 1 → risorsa disponibile,
- valore 0 → risorsa occupata.

Funzionamento:

- ingresso in chirurgia → risorse occupate,
- uscita → risorse rilasciate.

Questo garantisce mutua esclusione.

4.6 Sincronizzazione e cicli di attesa

Un elemento fondamentale del modello è rappresentato dai canali di sincronizzazione, utilizzati per coordinare gli automi e gestire correttamente l'accesso alle risorse.

In UPPAAL, i canali permettono di sincronizzare le transizioni tra più automi tramite operazioni di invio (`"!"`) e ricezione (`"?"`).

4.6.1 Canali utilizzati

Nel modello sono presenti i seguenti canali principali:

- `resSurg!` / `resSurg?`: utilizzato per segnalare il rilascio delle risorse chirurgiche;
- `resRehab!` / `resRehab?`: utilizzato per segnalare il rilascio del robot di riabilitazione.

Quando un automa esegue una transizione con `!`, un altro automa sincronizzato con `?` può eseguire simultaneamente la transizione corrispondente.

4.6.2 Ciclo tra MultiChoices e WaitingSurgery

Il comportamento di attesa per l'accesso alla chirurgia è modellato tramite un ciclo tra le locazioni `MultiChoices` e `WaitingSurgery`.

Funzionamento:

- Il paziente entra in `MultiChoices` (stato committed) e valuta immediatamente le condizioni:
  - disponibilità delle risorse,
  - rispetto del vincolo temporale,
  - eventuale priorità (EM vs PL).
- Se le condizioni sono soddisfatte, il paziente entra in `InSurgery`.
- Se le risorse non sono disponibili, il paziente passa in `WaitingSurgery`.

Nello stato `WaitingSurgery`:

- il tempo continua ad avanzare;
- il clock monitora il rispetto della deadline;
- il paziente resta in attesa finché non viene notificata la disponibilità delle risorse.

Il ritorno verso `MultiChoices` avviene tramite il canale:

`resSurg?`

che viene attivato quando un altro paziente termina la chirurgia ed esegue:

`resSurg!`

Questo meccanismo implementa una **riattivazione reattiva**:

- i pazienti in attesa non controllano continuamente le risorse;
- vengono "risvegliati" solo quando una risorsa si libera.

4.6.3 Ciclo tra MultiChoicesRehab e WaitingRehab

Un meccanismo analogo è utilizzato per la fase di riabilitazione.

Funzionamento:

- dopo la chirurgia, il paziente entra nello stato `MultiChoicesRehab`;
- se il robot di riabilitazione è disponibile, accede direttamente a `InRehab`;
- altrimenti passa in `WaitingRehab`.

Durante l'attesa:

- il paziente rimane bloccato fino alla disponibilità della risorsa;
- il ritorno alla decisione avviene tramite sincronizzazione.

Anche in questo caso, il ciclo è governato dal canale:

`resRehab?`

attivato dal rilascio della risorsa:

`resRehab!`

4.6.4 Interpretazione del modello

Questa struttura introduce un comportamento molto importante:

- evita il polling continuo delle risorse;
- modella correttamente la concorrenza tra pazienti;
- garantisce che le decisioni vengano rivalutate solo quando necessario.

Dal punto di vista dei sistemi real-time, questo è equivalente a un meccanismo **event-driven**, in cui i task vengono riattivati al verificarsi di eventi (rilascio risorsa) anziché controllare continuamente lo stato del sistema.

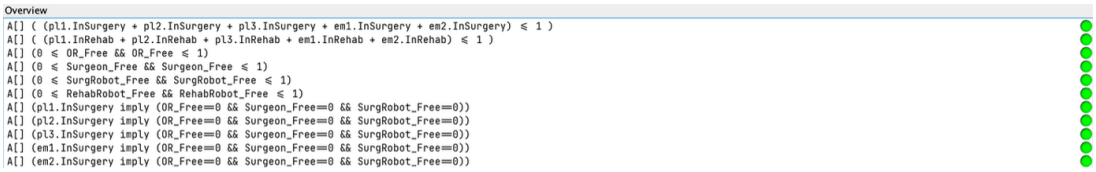
Questo approccio migliora:

- l'efficienza del modello,
- la fedeltà rispetto al comportamento reale,
- la chiarezza nella verifica formale.

4.7 Verifica tramite query

Le query permettono di verificare formalmente il sistema utilizzando la logica temporale di UPPAAL.

4.7.1 Mutua esclusione



Query 1: Mutua esclusione in chirurgia

$$A[]\ ((pl1.InSurgery + pl2.InSurgery + pl3.InSurgery + em1.InSurgery + em2.InSurgery) \leq 1)$$

Questa query verifica che in ogni stato del sistema al più un paziente possa trovarsi nello stato `InSurgery`.

- garantisce l'utilizzo esclusivo della sala operatoria;
- impedisce accessi concorrenti alla chirurgia;
- modella correttamente la capacità unitaria della risorsa.

Query 2: Mutua esclusione in riabilitazione

$$A[]\ ((pl1.InRehab + pl2.InRehab + pl3.InRehab + em1.InRehab + em2.InRehab) \leq 1)$$

Verifica che anche il robot di riabilitazione sia utilizzato da un solo paziente alla volta.

Query 3–6: Coerenza delle risorse

$$A[]\ (0 \leq OR_Free \leq 1)$$
$$A[]\ (0 \leq Surgeon_Free \leq 1)$$
$$A[]\ (0 \leq SurgRobot_Free \leq 1)$$
$$A[]\ (0 \leq RehabRobot_Free \leq 1)$$

Queste query verificano che le variabili che rappresentano le risorse siano sempre binarie (libero/occupato).

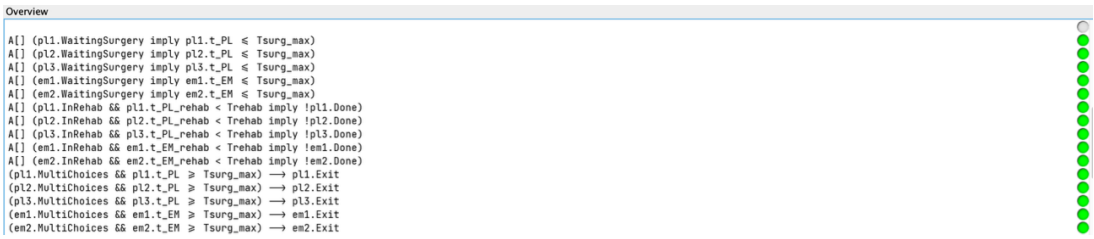
Query 7–11: Coerenza tra stato e risorse
Esempio:

$$A[]\ (pl1.InSurgery \Rightarrow (OR_Free == 0 \wedge Surgeon_Free == 0 \wedge SurgRobot_Free == 0))$$

(e analoghe per tutti i pazienti)
Queste query verificano che:

- quando un paziente è in chirurgia, tutte le risorse siano occupate;
- il modello sia consistente dal punto di vista logico;
- non ci siano stati incoerenti.

4.7.2 Vincoli temporali



Query 12–16: Deadline chirurgia
Esempio:

$$A[]\ (pl1.WaitingSurgery \Rightarrow pl1.t_PL \leq Tsurg_max)$$

(e analoghe per tutti i pazienti)
Queste query verificano che:

- il tempo di attesa non superi il limite massimo;
- i pazienti rispettino le deadline cliniche;
- non esistano attese indefinite.

Query 17–21: Vincoli sulla riabilitazione
Esempio:

$$A[]\ (pl1.InRehab \wedge pl1.t_PL_rehab < Trehab \Rightarrow \neg pl1.Done)$$

Queste query garantiscono che:

- un paziente non possa completare la riabilitazione prima del tempo previsto;
- la durata della riabilitazione sia rispettata;
- il comportamento temporale sia realistico.

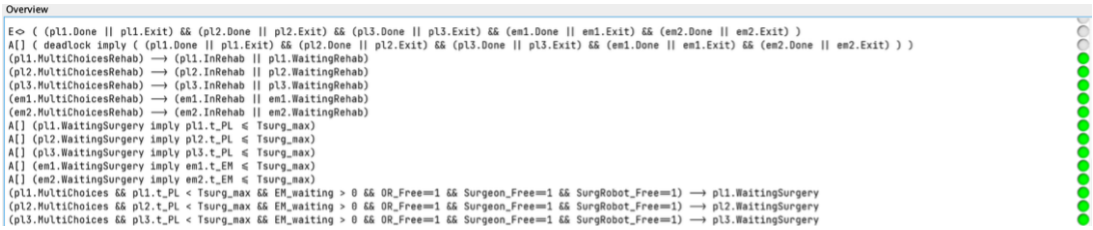
Query 22–26: Violazione deadline → uscita
Esempio:

$$(pl1.MultiChoices \wedge pl1.t_PL \geq Tsurg_max) \rightarrow pl1.Exit$$

Queste query verificano che:

- se la deadline viene superata, il paziente esce dal sistema;
- il modello gestisce correttamente i casi limite;
- non si creano situazioni non gestite.

4.7.3 Correttezza globale



Query 27: Raggiungibilità stato finale

$$E <> \ ((pl1.Done \vee pl1.Exit) \wedge (pl2.Done \vee pl2.Exit) \wedge (pl3.Done \vee pl3.Exit) \wedge (em1.Done \vee em1.Exit) \wedge (e$$

Questa query verifica che esista almeno un'esecuzione in cui tutti i pazienti terminano il processo (completamento o uscita).

Query 28: Assenza di deadlock

$$A[] \ \neg deadlock$$

Garantisce che:

- il sistema non si blocchi mai;
- esista sempre almeno una transizione attivabile;
- le risorse vengano correttamente rilasciate.

Query 29–33: Correttezza delle transizioni
Esempio:

$$(pl1.MultiChoicesRehab) \rightarrow (pl1.InRehab \vee pl1.WaitingRehab)$$

Queste query verificano che:

- le transizioni siano corrette;
- il flusso del paziente sia consistente;
- non esistano stati "bloccati" intermedi.

Query finali: priorità e attesa
Esempio:

$$(pl1.MultiChoices \wedge pl1.t_PL < Tsurg_max \wedge EM_waiting > 0 \wedge risorse\,disponibili) \rightarrow pl1.Waiting$$

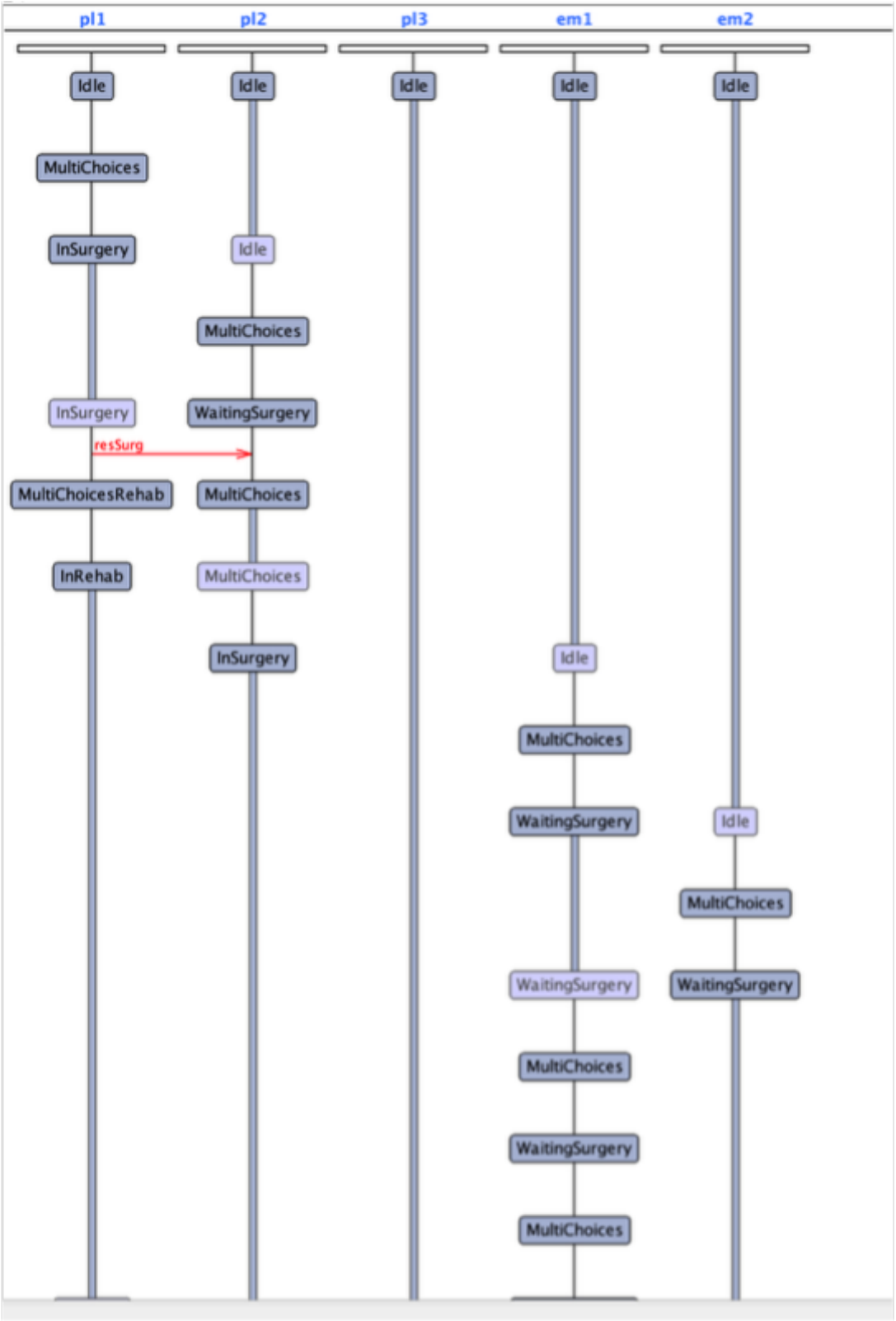
Queste query modellano la priorità:

- i pazienti EM hanno precedenza sui PL;
- i pazienti programmati attendono in presenza di emergenze;
- il sistema implementa una politica di scheduling con priorità.

4.8 Analisi dettagliata della simulazione

La simulazione del modello in UPPAAL consente di osservare l'evoluzione dinamica del sistema e di verificare il comportamento concorrente dei pazienti in presenza di risorse condivise e vincoli temporali.

4.8.1 Configurazione iniziale



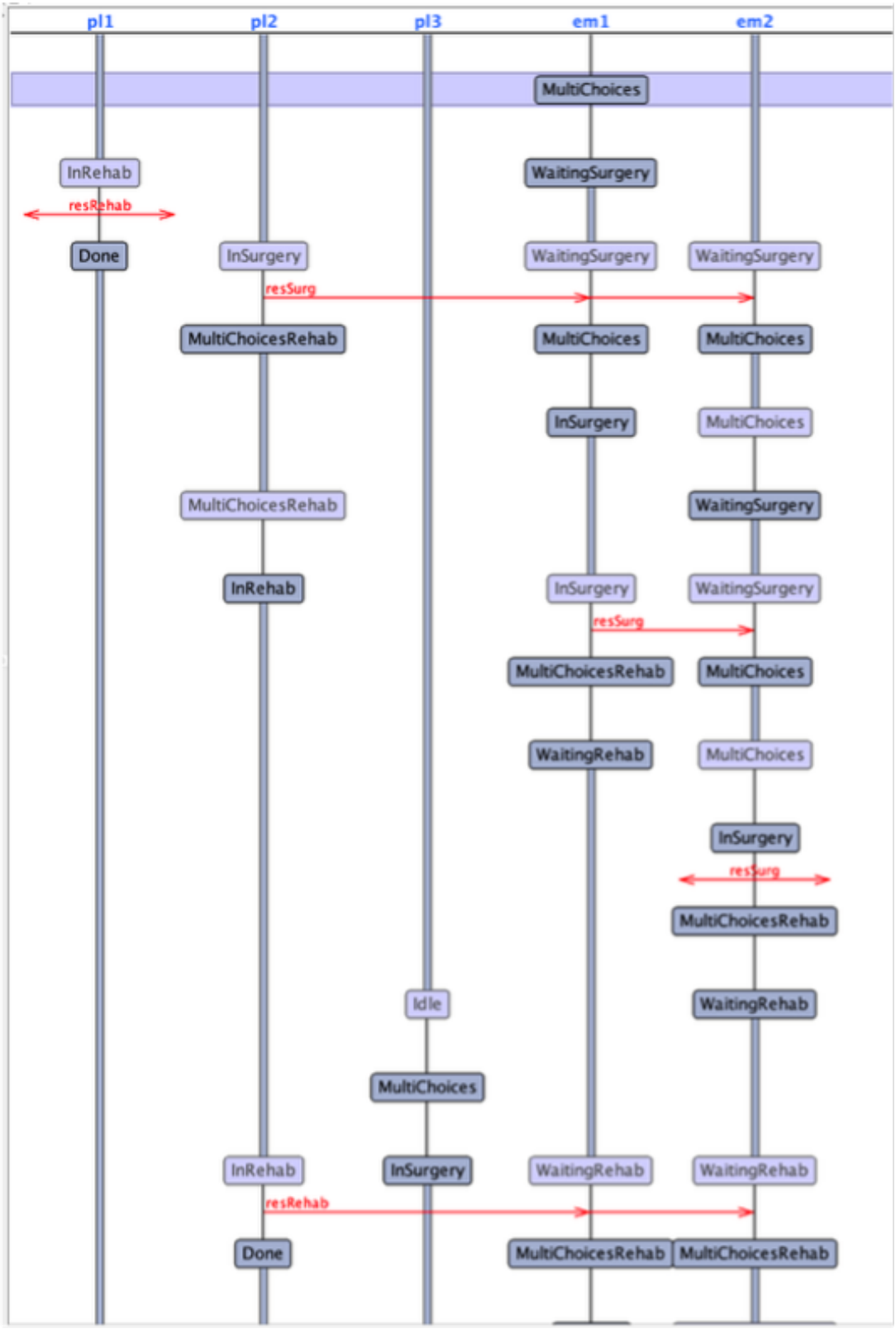
Nello stato iniziale:

- tutti i pazienti si trovano nella locazione iniziale (`Idle`);

- tutte le risorse (`OR_Free`, `Surgeon_Free`, `SurgRobot_Free`, `RehabRobot_Free`) sono disponibili;
- i clock sono inizializzati a zero.

Successivamente, i pazienti entrano nella locazione `MultiChoices`, che essendo committed forza una scelta immediata senza avanzamento del tempo.

4.8.2 Evoluzione del sistema e competizione per le risorse



Durante l'evoluzione del sistema si osserva che:

- solo un numero limitato di pazienti può accedere alla chirurgia, a causa della disponibilità limitata delle risorse;

- i pazienti che non trovano risorse disponibili rimangono nello stato `WaitingSurgery`;
- i clock associati ai pazienti continuano ad aumentare durante l'attesa.

Quando un paziente entra nello stato `InSurgery`:

- le risorse chirurgiche vengono occupate (valore 0);
- nessun altro paziente può accedere alla chirurgia finché le risorse non vengono rilasciate.

Al termine della chirurgia:

- le risorse vengono rilasciate tramite sincronizzazione (`resSurg`);
- altri pazienti possono essere abilitati ad entrare in chirurgia.

Questo evidenzia chiaramente il comportamento concorrente e la competizione tra i pazienti.

4.8.3 Fase di riabilitazione

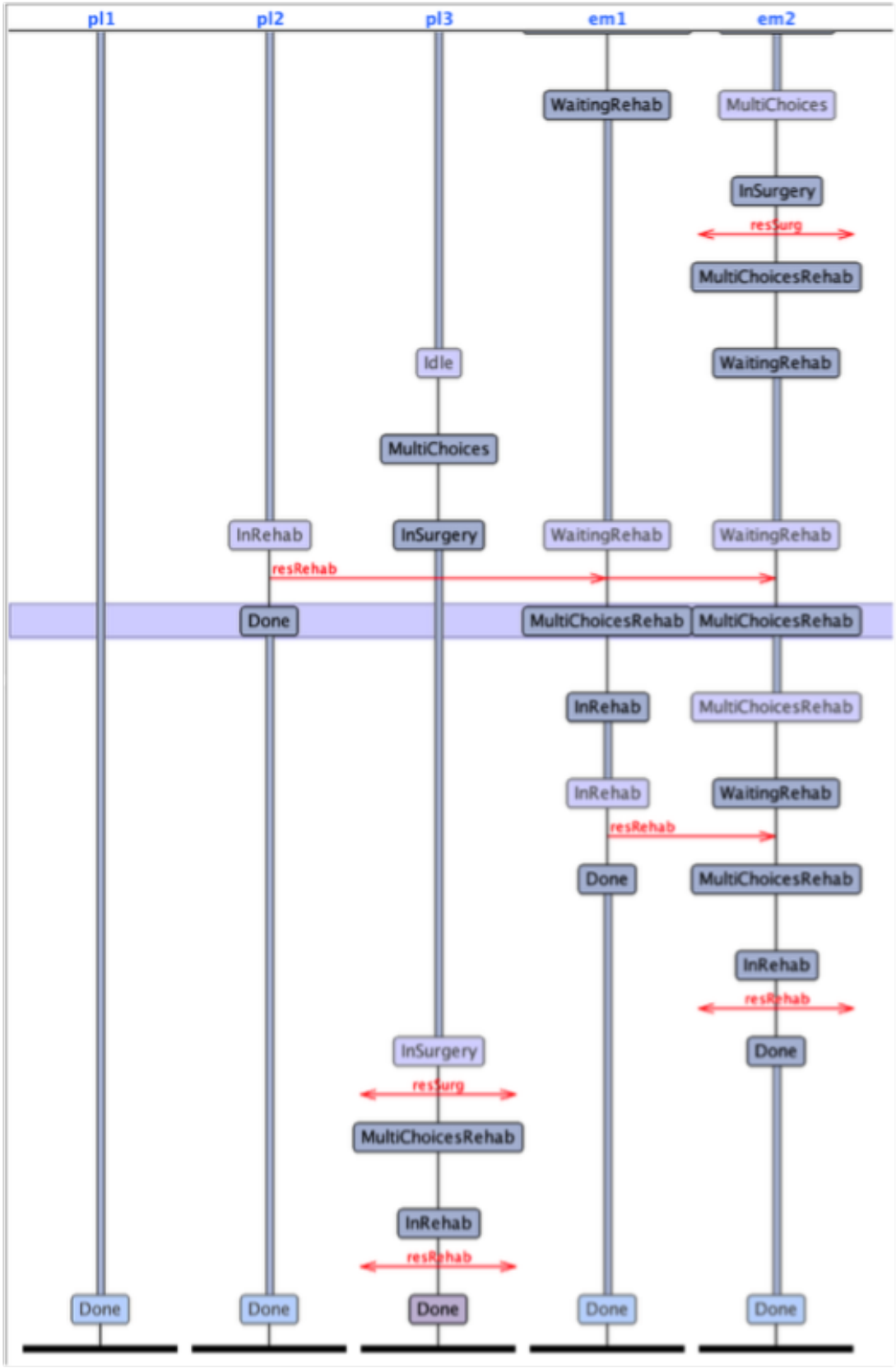
Dopo la chirurgia:

- i pazienti passano nello stato `WaitingRehab`;
- l'accesso alla riabilitazione è regolato dalla disponibilità del robot;
- il clock associato alla riabilitazione controlla la durata del processo.

Analogamente alla chirurgia, anche in questa fase si osserva:

- competizione per le risorse,
- sincronizzazione tra automi,
- utilizzo esclusivo del robot di riabilitazione.

4.8.4 Completamento del sistema



Nella configurazione finale:

- tutti i pazienti raggiungono la locazione `Done`;
- tutte le risorse risultano nuovamente disponibili;
- non si verificano situazioni di deadlock.

4.8.5 Osservazioni principali

Dalla simulazione emergono i seguenti aspetti:

- il sistema gestisce correttamente la concorrenza tra pazienti;
- le risorse sono utilizzate in modo esclusivo;
- il rilascio delle risorse abilita dinamicamente altri pazienti;
- i vincoli temporali sono rispettati durante l'intero processo.

4.9 Conclusione

Gli automi temporizzati permettono di modellare e verificare i vincoli temporali del sistema, integrando la dimensione temporale con la gestione della concorrenza.

In combinazione con le reti di Petri, consentono una rappresentazione completa del comportamento del sistema.

Capitolo 5

Scheduling real-time del sistema

Il sistema ospedaliero può essere modellato come un problema di scheduling real-time, in cui:

- i pazienti rappresentano task concorrenti,
- le risorse (sala operatoria, chirurgo, robot) sono limitate,
- esiste un vincolo temporale sull'inizio della chirurgia.

Ogni paziente è modellato come un task aperiodico $\tau_i = (r_i, C_i, d_i)$, dove:

- r_i è il tempo di arrivo,
- C_i è il tempo di esecuzione,
- d_i è la deadline di inizio chirurgia.

La deadline è definita come:

$$d_i = r_i + T_{surg_max}$$

Caratteristiche del sistema:

- task aperiodici,
- scheduling dinamico (decisioni online),
- vincolo di non-preemption,
- priorità cliniche (EM > PL),
- risorse condivise.

5.1 Modellazione del sistema come problema real-time

Il sistema è assimilabile a uno scheduling su singola risorsa:

- la sala operatoria è vista come un processore,
- può essere eseguito un solo task alla volta.

Osservazione fondamentale:

- la deadline riguarda l'inizio del task,
- non il completamento.

Un task va in *deadline miss* se:

$$t_{start} > d_i$$

Il problema consiste quindi nel garantire l'accesso alla risorsa entro un tempo massimo.

5.2 Politiche di scheduling

5.2.1 Fixed Priority Scheduling (FPS)

Assegna priorità fisse ai task e seleziona sempre quello con priorità più alta.

Limiti:

- non considera le deadline,
- non adatto a task aperiodici.

5.2.2 Earliest Deadline First (EDF)

Seleziona il task con deadline più vicina:

$$d_i = r_i + T_{surg_max}$$

Proprietà:

- politica dinamica,
- ottimale in sistemi preemptive.

Limiti:

- non considera priorità cliniche,
- meno efficace in sistemi non-preemptive.

5.2.3 Least Slack Time (LST)

Seleziona il task con slack minimo:

$$slack_i = d_i - t - C_i$$

Limite principale: richiede preemption.

5.2.4 Non-Preemptive Scheduling

- un task non può essere interrotto,
- le decisioni sono prese solo quando la risorsa si libera.

È il caso del sistema chirurgico reale.

5.3 Confronto tra le politiche

Sintesi:

- FPS → non adatto (ignora le deadline),
- LST → non adatto (richiede preemption),
- EDF → adatto ma incompleto,
- non-preemptive → vincolo strutturale.

5.4 Scelta della politica di scheduling

Si adotta una politica ibrida:

Non-Preemptive EM-first EDF

Criterio:

- priorità assoluta ai pazienti EM,
- EDF all'interno della stessa classe,
- decisioni solo a risorsa libera.

Questa scelta garantisce coerenza clinica e rispetto delle deadline.

5.5 Analisi della schedulabilità

Per valutare la fattibilità dello scheduling real-time del sistema, si introduce il fattore di utilizzazione della CPU:

$$U = \sum_{i=1}^{n} \frac{C_i}{T_i}$$

dove C_i rappresenta il tempo di esecuzione del task i e T_i il suo periodo (o tempo minimo tra due arrivi successivi nel caso di task sporadici).

Tale grandezza misura il carico complessivo del sistema e costituisce una condizione fondamentale per la schedulabilità.

5.5.1 Condizione per EDF

Nel caso di scheduling con priorità dinamiche basato su Earliest Deadline First (EDF), utilizzato nel presente progetto, vale la seguente proprietà:

- il sistema è schedulabile se e solo se:

$$U \leq 1$$

Questa rappresenta una condizione sia necessaria che sufficiente, il che rende EDF un algoritmo ottimale per sistemi real-time su singolo processore.

5.5.2 Confronto con il bound di Liu & Layland

Per completezza teorica, si richiama il bound di Liu & Layland, valido per algoritmi a priorità fissa come Rate Monotonic (RM):

$$U \leq n(2^{1/n} - 1)$$

Tale condizione è sufficiente ma non necessaria e, per un numero elevato di task, tende al valore:

$$\ln 2 \approx 0.69$$

Ciò implica che, con RM, la schedulabilità è garantita solo se l'utilizzazione della CPU è inferiore a circa il 69%.

Tuttavia, questo bound non si applica al caso in esame, poiché il sistema è modellato mediante EDF, che consente di sfruttare completamente la capacità della CPU fino al limite teorico $U = 1$.

5.5.3 Interpretazione nel contesto del sistema

Nel sistema ospedaliero modellato:

- i pazienti sono interpretati come task aperiodici/sporadici,
- le risorse (sala operatoria, robot, personale) introducono vincoli aggiuntivi,
- lo scheduling è non-preemptive.

Pertanto, la condizione $U \leq 1$ rappresenta una stima teorica ideale. Nella pratica, ulteriori fattori come:

- tempi di blocco,
- accesso a risorse condivise,
- vincoli di non-preemption

possono ridurre la schedulabilità effettiva del sistema.

Nonostante ciò, l'utilizzo di EDF rimane una scelta appropriata in quanto massimizza l'utilizzo delle risorse e consente una gestione efficiente delle deadline dei pazienti, in particolare per quelli con priorità clinica elevata (emergenze).

5.6 Esempio numerico di scheduling

Si considerino:

Task	Classe	r_i	C_i	d_i
τ_1	PL	0	4	6
τ_2	EM	1	2	7
τ_3	PL	2	1	8

Ordine di esecuzione:

$$\tau_1 \rightarrow \tau_2 \rightarrow \tau_3$$

Tutte le deadline risultano rispettate.

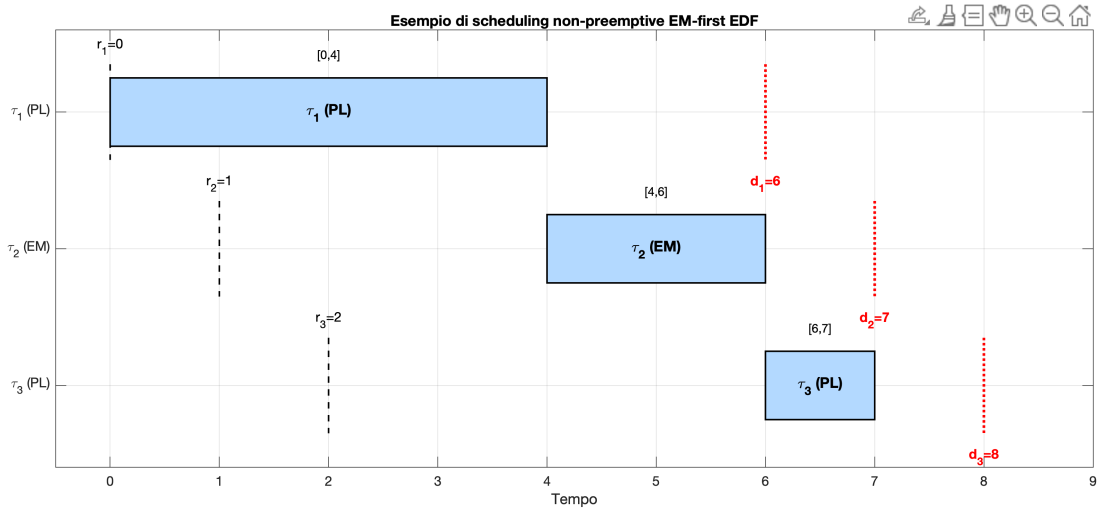


Figura 5.1: Scheduling non-preemptive EM-first EDF.

Il diagramma di Gantt mostra l'ordine temporale con cui i task vengono eseguiti sulla risorsa. Ogni blocco rappresenta l'intervallo di occupazione della sala operatoria da parte di un paziente. In questo caso si osserva che, pur essendo τ_2 arrivato dopo τ_1, esso viene eseguito prima di τ_3 grazie alla priorità clinica EM. Inoltre, l'assenza di sovrapposizioni tra i blocchi riflette il vincolo di non-preemption: una volta iniziato, un task deve terminare prima che il successivo possa essere avviato.

5.6.1 Caso di overload

Per evidenziare il comportamento del sistema in condizioni critiche, si considera un secondo esempio in cui il carico computazionale è elevato.

Task considerati:

Task	r_i	C_i	d_i
τ_1	0	4	5
τ_2	1	4	6
τ_3	2	4	7

In questo caso, il sistema non è in grado di rispettare tutte le deadline.

Evoluzione temporale:

- τ_1 viene eseguito da $t = 0$ a $t = 4$,
- τ_2 inizia a $t = 4$ (deadline $d_2 = 6$),
- τ_3 inizia a $t = 8$ (deadline $d_3 = 7$).

Osservazione:

- τ_3 viola la deadline,
- il sistema si trova in condizione di overload.

Andamento dei deadline miss

Per analizzare la dinamica delle violazioni, si considera il numero cumulativo di deadline miss nel tempo.

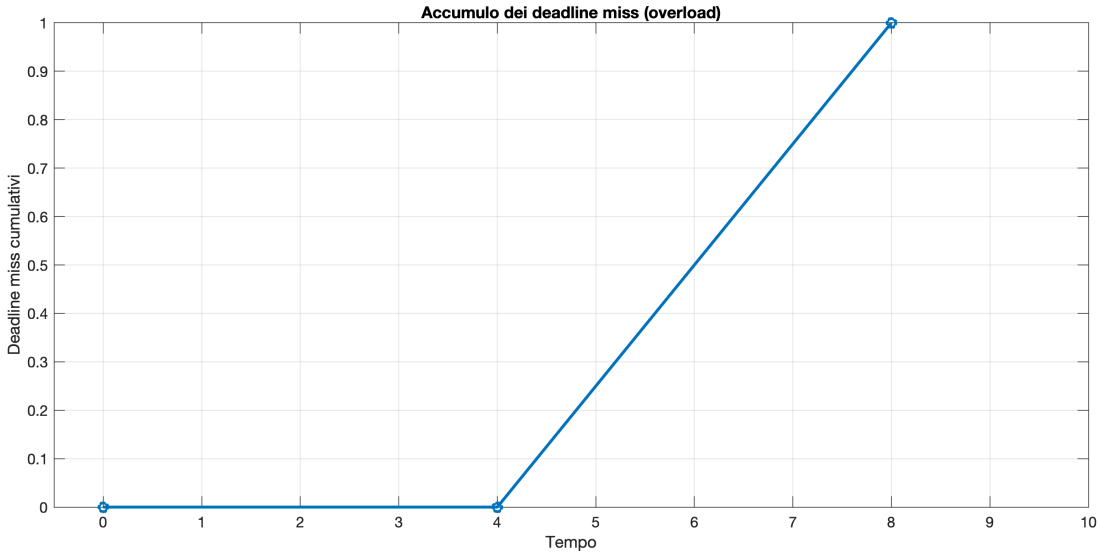


Figura 5.2: Accumulo dei deadline miss in condizioni di overload.

La Figura 5.2 mostra l'andamento cumulativo dei deadline miss nel tempo. Se la curva rimane costante significa che, in quell'intervallo, non si sono verificate nuove violazioni. Al contrario, ogni incremento della curva corrisponde alla comparsa di almeno un task che ha iniziato oltre la propria deadline. Una crescita progressiva indica che il sistema non riesce a gestire il carico richiesto e che le violazioni tendono ad accumularsi nel tempo, evidenziando una progressiva saturazione delle risorse.

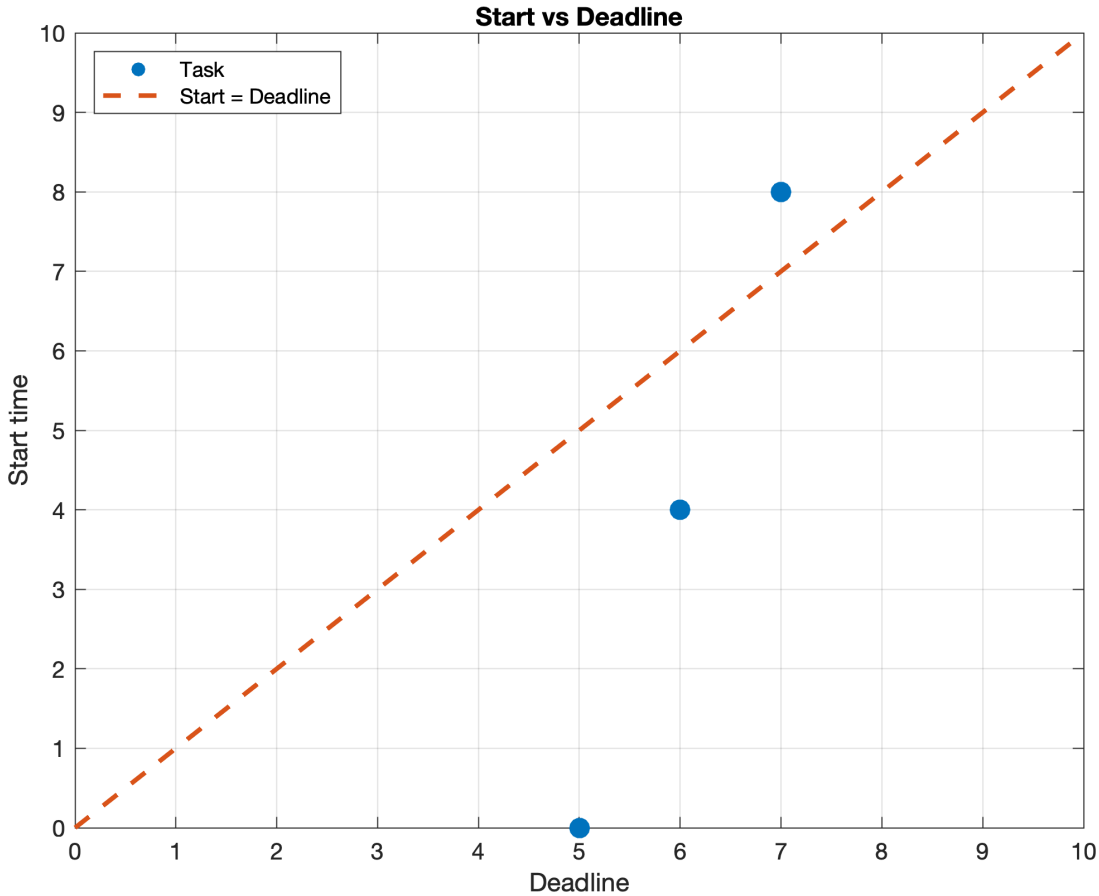


Figura 5.3: Grafico start vs deadline in condizioni di overload.

Nel grafico ogni punto rappresenta un task e mette in relazione il suo tempo di inizio con la relativa deadline. La diagonale rappresenta la condizione limite $t_{start} = d_i$. I punti al di sotto della diagonale identificano task che iniziano in tempo, mentre i punti al di sopra della diagonale rappresentano task che iniziano oltre la propria deadline e quindi generano un deadline miss. Questo grafico è utile perché consente di visualizzare immediatamente il margine con cui le deadline vengono rispettate oppure violate. La presenza di punti sopra la diagonale conferma chiaramente la condizione di overload del sistema.

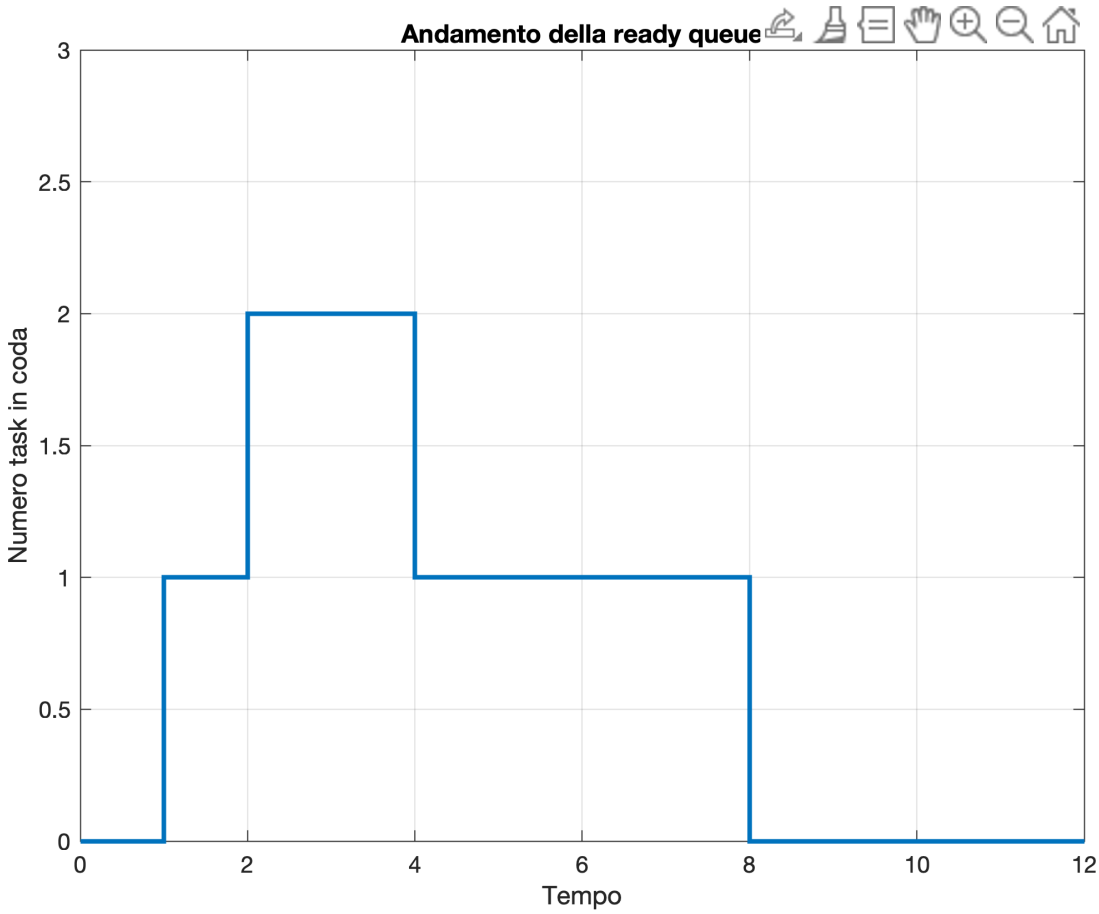


Figura 5.4: Andamento della ready queue nel caso di overload.

Questo grafico mostra il numero di task in attesa nel tempo. La coda cresce quando arrivano nuovi task mentre la risorsa è occupata, e diminuisce quando un task termina e ne viene avviato un altro. L'andamento a gradini riflette il comportamento non-preemptive del sistema

5.7 Esempio comparativo su tempi di attesa

Per rendere più esplicito l'effetto delle politiche di scheduling sui tempi di attesa, si considera un insieme più ampio di pazienti appartenenti alle classi EM e PL.

L'obiettivo non è effettuare una simulazione completa del sistema, ma mostrare, in modo illustrativo, come strumenti grafici quali boxplot e funzioni di distribuzione cumulativa (CDF) possano evidenziare le differenze tra le due politiche considerate:

- Non-Preemptive EDF,
- EM-first EDF.

Nel caso della politica NP-EDF, i tempi di attesa risultano distribuiti in modo più uniforme tra le due classi di pazienti. Al contrario, con EM-first EDF i pazienti in emergenza tendono a presentare attese più contenute, mentre i pazienti programmati risultano mediamente penalizzati.

5.7.1 Boxplot dei tempi di attesa

Il boxplot consente di visualizzare la distribuzione dei tempi di attesa evidenziando mediana, quartili e dispersione. Nel caso analizzato, la politica EM-first EDF mostra una riduzione significativa dei tempi per i pazienti in emergenza, a scapito di un aumento delle attese per i pazienti programmati, evidenziando il trade-off tra priorità clinica ed equità.

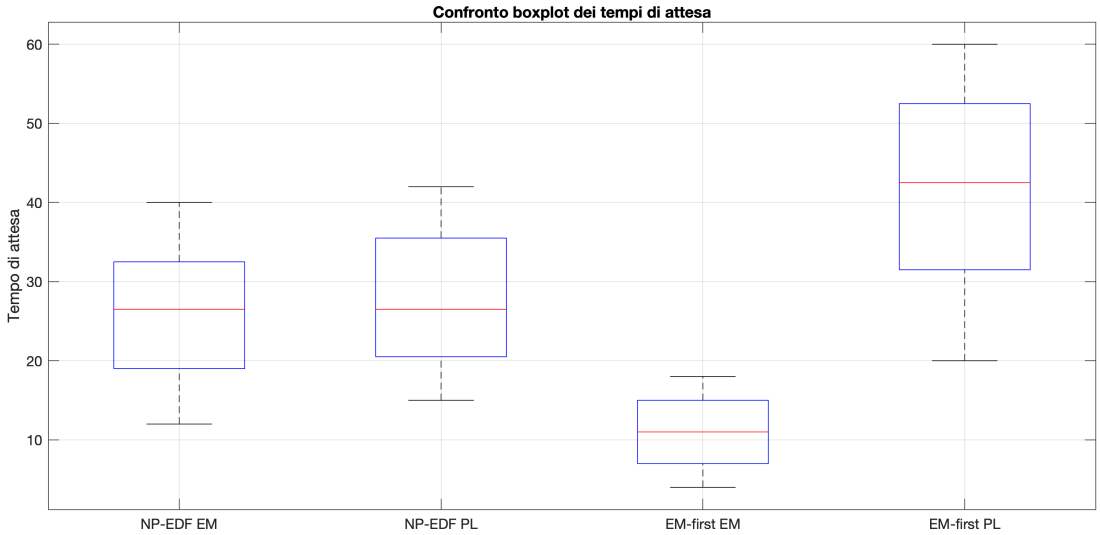


Figura 5.5: Confronto dei tempi di attesa mediante boxplot.

La Figura 5.5 mostra la distribuzione dei tempi di attesa per pazienti EM e PL nelle due politiche. Si osserva che la politica EM-first EDF riduce i tempi di attesa dei pazienti in emergenza, a scapito di un aumento delle attese per i pazienti programmati.

Boxplot dei tempi di attesa

Il boxplot serve a riassumere in modo compatto come sono distribuiti i tempi di attesa in ciascun gruppo di pazienti. Non mostra solo un valore medio, ma dà informazioni sulla posizione tipica dei dati e sulla loro dispersione.

In particolare:

- la linea centrale del box indica la mediana del tempo di attesa, cioè il valore che divide a metà i dati: il 50% dei pazienti ha un'attesa inferiore a quel valore e il restante 50% ha un'attesa superiore;
- il bordo inferiore del box corrisponde al primo quartile (Q_1), mentre il bordo superiore corrisponde al terzo quartile (Q_3);
- il box rappresenta quindi l'intervallo interquartile, cioè la fascia in cui cade il 50% centrale dei dati;
- i baffi indicano l'estensione dei valori osservati al di fuori del box, cioè quanto i tempi di attesa si allontanano dai valori centrali.

Come si legge in pratica:

- se un box è più in basso sull'asse verticale, significa che quel gruppo ha tempi di attesa generalmente minori;

- se la mediana è più bassa, significa che il valore "tipico" dell'attesa è più favorevole;
- se il box è più piccolo, i tempi sono più concentrati e meno variabili;
- se il box o i baffi sono molto estesi, significa che le attese sono più disperse e meno uniformi.

Interpretazione del grafico:

- nella politica EM-first EDF, il box dei pazienti EM tende a collocarsi più in basso, indicando attese generalmente ridotte;
- nella stessa politica, il box dei pazienti PL tende invece a spostarsi verso valori più alti, segnalando un peggioramento dei tempi di attesa;
- nella politica NP-EDF, i box delle due classi risultano più vicini, il che suggerisce una distribuzione più equilibrata delle attese.

In altre parole, il boxplot fa vedere immediatamente il compromesso introdotto dalla politica EM-first EDF: i pazienti in emergenza vengono favoriti, mentre quelli programmati attendono mediamente di più.

5.7.2 CDF dei tempi di attesa

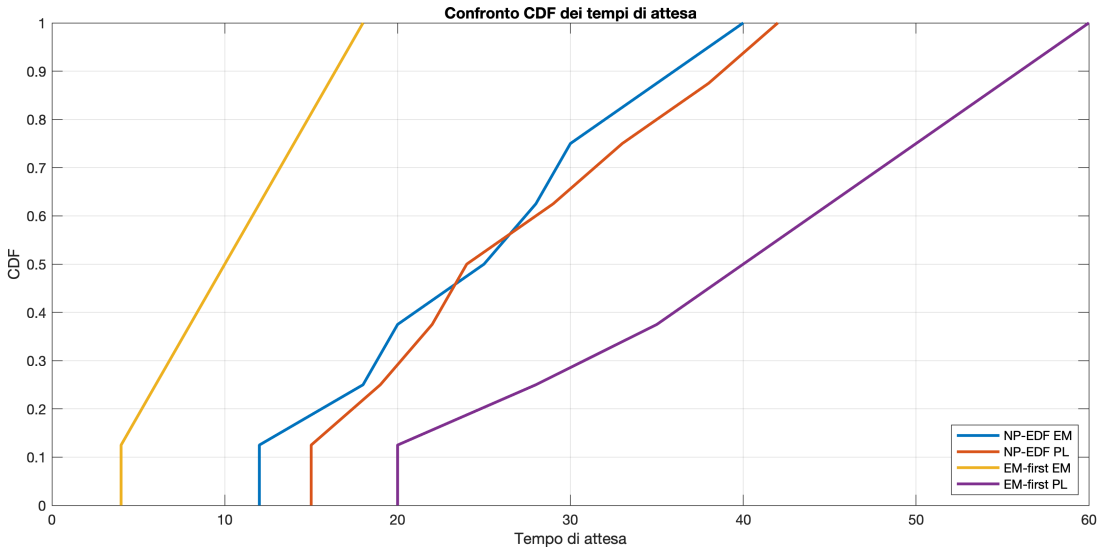


Figura 5.6: Confronto delle CDF dei tempi di attesa.

La Figura 5.6 riporta le funzioni di distribuzione cumulativa dei tempi di attesa. Curve più spostate verso sinistra indicano una maggiore probabilità di osservare tempi di attesa contenuti. In particolare, la politica EM-first EDF mostra una protezione temporale più marcata per i pazienti EM, mentre i pazienti PL presentano una distribuzione più sfavorevole.

CDF dei tempi di attesa

La funzione di distribuzione cumulativa (CDF) rappresenta la probabilità che il tempo di attesa sia inferiore o uguale a un determinato valore.

Formalmente:

$$F(t) = P(\text{tempo di attesa} \leq t)$$

Criterio di interpretazione:

- per un dato valore sull'asse orizzontale, la CDF indica la frazione di pazienti con tempo di attesa inferiore a tale valore,
- curve più spostate verso sinistra indicano prestazioni migliori (attese più contenute),
- curve più ripide indicano una distribuzione più concentrata dei tempi di attesa.

Osservazione:

- nella politica EM-first EDF, la curva relativa ai pazienti EM risulta più a sinistra, indicando una maggiore probabilità di attese ridotte,
- al contrario, i pazienti PL presentano una curva spostata verso destra, evidenziando attese più elevate,
- nella politica NP-EDF, le curve risultano più simili, indicando una distribuzione più uniforme dei tempi di attesa.

Considerazioni

L'analisi congiunta di boxplot e CDF evidenzia il trade-off tra:

- equità tra i pazienti,
- rispetto delle priorità cliniche.

In particolare, la politica EM-first EDF privilegia i pazienti in emergenza, migliorandone le prestazioni temporali, a scapito di un peggioramento per i pazienti programmati.

5.8 Accesso alle risorse condivise

Le risorse sono:

- limitate,
- utilizzate in mutua esclusione.

Problemi principali:

- inversione di priorità,
- deadlock,
- ritardi non prevedibili.

5.9 Priority Inversion Problem

Si verifica quando un task a bassa priorità blocca uno ad alta priorità.

5.9.1 Esempio con tre task

Task	Priorità	Arrivo	C_i	Risorsa
τ_L	bassa	0	4	sì
τ_H	alta	1	2	sì
τ_M	media	2	2	no

Osservazione:

- τ_M non usa la risorsa ma ritarda τ_H,
- si ha inversione aggravata.

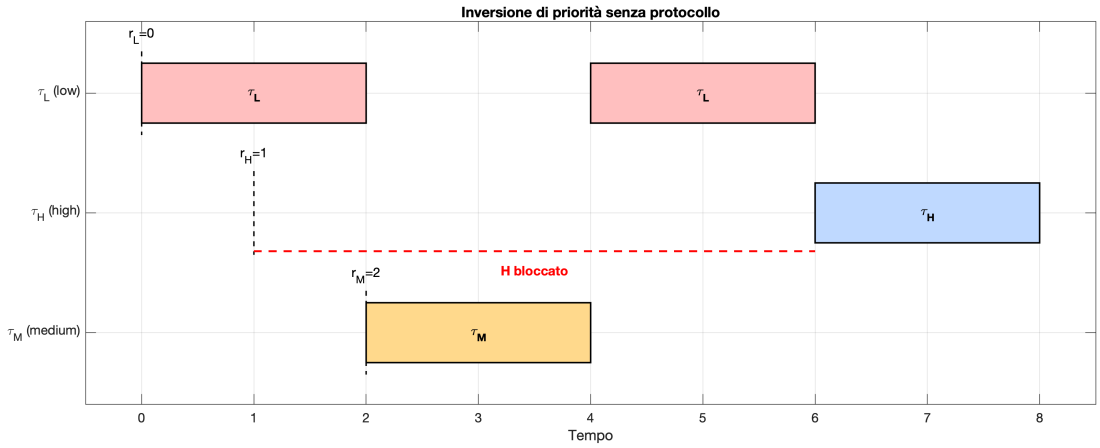


Figura 5.7: Inversione di priorità senza protocollo.

Il grafico mostra il fenomeno dell'inversione di priorità, che si verifica quando un task a bassa priorità blocca indirettamente l'esecuzione di un task ad alta priorità. In particolare, il task τ_L occupa la risorsa condivisa, impedendo al task τ_H di accedervi. Successivamente, l'arrivo del task τ_M, pur non utilizzando la risorsa, ritarda ulteriormente τ_L, prolungando il tempo di attesa di τ_H. Questo comportamento è particolarmente critico in sistemi real-time, poiché introduce ritardi non prevedibili nei task ad alta priorità. Nel contesto ospedaliero, ciò corrisponde a un paziente in emergenza (EM) che non può accedere alla chirurgia a causa della presenza di pazienti programmati (PL) e interferenze di altri processi.

5.10 Priority Inheritance Protocol

Il PIP è un meccanismo **reattivo**:

- il task a bassa priorità eredita temporaneamente quella alta,
- si riduce il tempo di blocco.

Limiti:

- non previene il deadlock,
- comportamento meno prevedibile.

5.10.1 Effetto sull'esempio

- τ_M non può preemptare τ_L,
- τ_H viene servito prima.

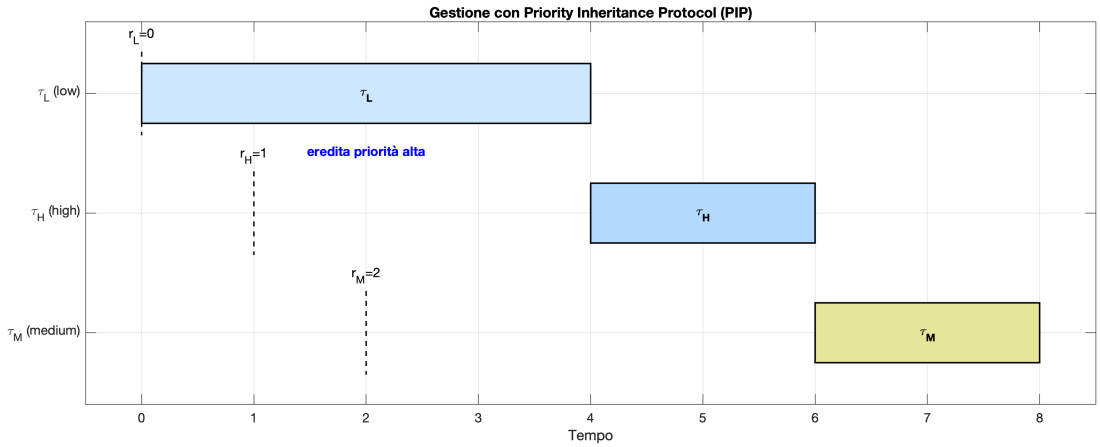


Figura 5.8: Effetto del PIP.

Il grafico illustra il funzionamento del Priority Inheritance Protocol (PIP). Quando il task ad alta priorità τ_H rimane bloccato a causa della risorsa occupata da τ_L, quest'ultimo eredita temporaneamente la priorità più alta. In questo modo, il task τ_M non può più interrompere τ_L, consentendo a τ_L di completare rapidamente l'utilizzo della risorsa e rilasciarla. Il protocollo riduce quindi il tempo di blocco del task ad alta priorità, mitigando l'inversione di priorità. Tuttavia, il PIP interviene in modo reattivo, ovvero solo dopo che il problema si è verificato, e non garantisce l'assenza di deadlock. Nel sistema ospedaliero, questo comportamento si traduce nel fatto che un paziente programmato, se sta utilizzando una risorsa critica, viene "favorito temporaneamente" per liberarla rapidamente e consentire l'accesso al paziente in emergenza.

5.11 Priority Ceiling Protocol

Il PCP è un protocollo **preventivo**.

- ogni risorsa ha un priority ceiling,
- l'accesso è controllato a priori.

Vantaggi:

- elimina inversioni non controllate,
- previene deadlock,
- garantisce tempi di blocco limitati.

5.11.1 Effetto sull'esempio

Il comportamento è simile al PIP, ma ottenuto in modo più controllato e prevedibile.

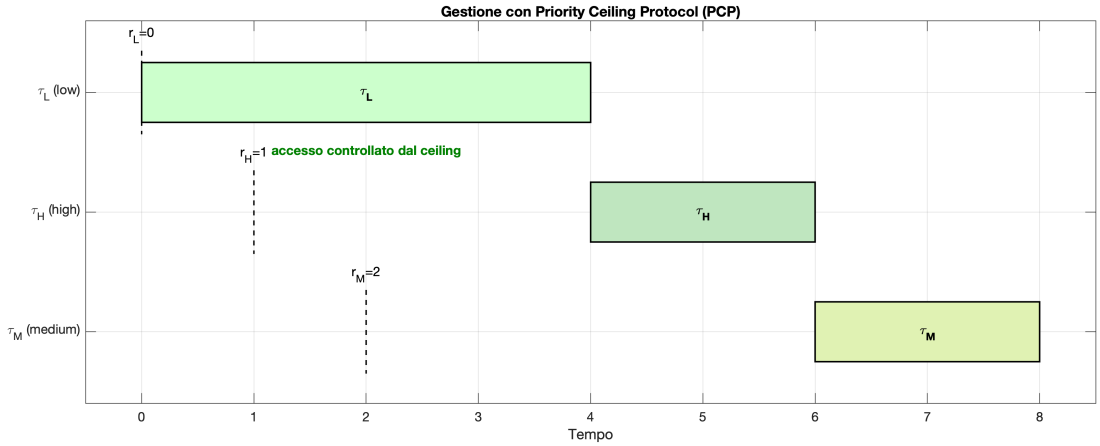


Figura 5.9: Effetto del PCP.

Il grafico rappresenta il comportamento del Priority Ceiling Protocol (PCP), che introduce un meccanismo preventivo per la gestione delle risorse condivise. A ciascuna risorsa è associata una priorità massima (ceiling), pari alla più alta tra i task che possono utilizzarla. Quando una risorsa è occupata, il protocollo impedisce a task che potrebbero causare interferenze di accedere al sistema, evitando a priori situazioni di inversione di priorità. Di conseguenza, il task τ_L può completare l'utilizzo della risorsa senza essere interrotto, mentre τ_H accede immediatamente alla risorsa non appena questa viene liberata. Rispetto al PIP, il PCP garantisce una maggiore prevedibilità e previene anche condizioni di deadlock. Nel contesto del sistema ospedaliero, ciò assicura che le risorse critiche, come la sala operatoria, siano gestite in modo sicuro e che i pazienti in emergenza possano accedervi senza ritardi non controllati.

5.12 Applicazione al sistema ospedaliero

- EM → alta priorità,
- PL → bassa priorità.

Il priority ceiling delle risorse è pari alla priorità EM.

5.13 Scelta del protocollo

Conclusione:

Priority Ceiling Protocol (PCP)

Motivazioni:

- maggiore robustezza,
- migliore prevedibilità,

- adatto a sistemi safety-critical.

Soluzione finale:

PCP + Non-Preemptive EM-first EDF

Capitolo 6

Conclusione

Questo progetto nasce dall'interesse verso l'applicazione delle tecniche di modellazione e analisi dei sistemi real-time in un contesto reale e critico come quello ospedaliero, in cui la corretta gestione delle risorse e il rispetto dei vincoli temporali sono fondamentali per garantire sicurezza ed efficienza. Attraverso l'utilizzo combinato di Reti di Petri, Automi Temporizzati (UPPAAL) e tecniche di scheduling real-time, è stato possibile modellare il sistema da diversi punti di vista: strutturale, temporale e decisionale. In particolare, sono state analizzate la concorrenza tra pazienti, la gestione delle risorse condivise e l'impatto delle politiche di scheduling sui tempi di attesa. I risultati ottenuti mostrano che: il sistema è corretto e privo di deadlock, le risorse sono gestite in mutua esclusione, è possibile garantire la raggiungibilità dello stato finale, le politiche di scheduling influenzano significativamente le prestazioni. L'analisi tramite boxplot e CDF ha evidenziato chiaramente il trade-off tra: priorità clinica (favorendo i pazienti in emergenza), equità del sistema (distribuzione uniforme delle attese). In conclusione, il progetto dimostra come l'integrazione di metodi formali e tecniche di scheduling consenta di progettare sistemi complessi in modo rigoroso, fornendo strumenti utili per supportare decisioni in contesti safety-critical come quello sanitario.